



MALTEPE UNIVERSITY

Faculty of Engineering and Natural Sciences - Software Engineering

SE 342

Software Validation and Testing

Instructor: Dr. Emre Athier Olca

Prepared by:

22 07 06 059 - Ceren Yaşar

22 07 06 028 - Mehmet Şenadlı

22 07 06 018 - Bilal Çiftçi

Table of Contents

REVISION HISTORY	3
Smart Attendance System	4
1. Introduction	4
1.1 Project Purpose	4
1.2 Scope	4
2. Requirements	4
2.1. Functional Requirements	4
2.2. Non-Functional Requirements (NFR)	5
3. System Design	7
3.1 ERD (Entity Relationship Diagram)	7
3.2 Database Schema Design (PostgreSQL)	7
3.3 UML Diagrams	10
3.3.1 Use Case Diagram	10
3.3.2 Sequence Diagrams	10
4. User Interface (UI) Design	12
4.1 Log in Page	13
4.2 Student Registration Screen	13
4.3 Automatic Attendance Session	14
4.4 Attendance Reports for Teacher	14
4.5 Student Reports Panel	14
5. Project Management & Methodology	15
5.1. Project Tracking with Jira	15
5.2 Sprint Planning	16
5.2.1 Sprint 0: Software Requirement and Analysis	16
5.2.2 Sprint 1: Database Design and Management	16
5.2.3 Sprint 2: Student Management and Registration Module	17
5.2.4 Sprint 3: Camera Integration System	17
5.2.5 Sprint 4: Reporting and Admin Panel	17
6. System Architecture & Technology Stack	18
6.1 Technology Stack	18
6.1.1 Project Management: Jira by Atlassian	18
6.1.2 Database Management: PostgreSQL	18
6.1.3 Frontend Framework: Why Next.js?	18
6.1.4 Backend & AI Engine: Why Python?	19
6.2 Version Control System	19
7. System Implementation	20
7.1 Teacher Interfaces	21
7.2 Student Interfaces	28
8. Test Development Process: Test Conditions, Test Cases and Test Procedures	30
8.1 Test Cases for Student Management Page	30
8.2 Test Cases for Course Registration and Course Enrollment	34
8.3 Test Cases for Instructor Management Page	41
9. Unit Testing Implementation	45
9.1 Frontend Tests	45
9.2 Backend Tests	55
10. Automatic Attendance System Based on Face Recognition	68
10.1 Technologies and Libraries Used	68

REVISION HISTORY

Version	Date	Description	Author
0.1	27.10.25	Initial setup of project documentation, including introduction, scope, and detailed requirements.	Ceren Yaşar
0.2	04.11.25	Definition of methodology, sprint planning, workflow structure, and task organization.	Ceren Yaşar
0.3	09.11.25	Addition of use case, sequence diagrams for system modeling.	Ceren Yaşar
0.4	16.11.25	Creation of UI/UX wireframes, layouts, and design components in Figma.	Bilal Çiftçi
0.5	23.11.25	Design of database schema, ER diagrams, and table structures.	Mehmet Şenadlı
0.6	30.11.25	Development of UI components, pages, and frontend logic based on the design and defined frontend technology stack.	Ceren Yaşar
0.7	07.12.25	Defined the backend technology stack, including core frameworks, libraries, and architectural choices.	All Team
0.8	14.12.25	Documented the test development process including test conditions, test cases, and test procedures in accordance with course materials.	Ceren Yaşar
0.9	21.12.25	Added unit tests for Student Enrollment functionality, covering backend API endpoints and frontend form rendering.	Ceren Yaşar
1.0	28.12.25	Added a face recognition–based automatic attendance feature using image processing and machine learning techniques.	All Team

Smart Attendance System

🔗 For detailed tasks and progress, see the Jira board:

<https://cerenysr1-1762092278993.atlassian.net/jira/software/projects/SCRUM/list>

🔗 For source code and repository structure, see the GitHub project:

<https://github.com/cerenyasarr/SE342-AttendanceSystem>

1. Introduction

1.1 Project Purpose

This system aims to automate the attendance process in an educational institution. The process begins with the teacher adding student records along with their photos to the system. The integrated camera system then detects student faces in the classroom and compares them with the registered photos. When a successful match occurs, attendance is automatically recorded, and a “Attendance recorded” notification is generated in the system.

1.2 Scope

The automated attendance system is developed to accelerate the attendance process and minimize errors in educational institutions. The system includes the following features:

- Adding student records to the system along with their photos,
- Recognizing and matching student faces in the classroom using the camera,
- Automatically recording attendance upon successful matches,
- Providing real-time notifications to teachers and administrators about the attendance status.

2. Requirements

2.1. Functional Requirements

FR-1: Student Management and Registration Module (SCRUM-25)

- **FR-1.1:** The system must allow the creation of new student registrations. (SCRUM-1)
- **FR-1.2:** A student registration form (name, number, department, class, photo) must be created. (SCRUM-5, 6)
- **FR-1.3:** Validation rules must be added for form fields. (SCRUM-7)
- **FR-1.4:** A student photo upload function must be developed. (SCRUM-8)

- **FR-1.5:** Uploaded photos must be stored in the system (database or file system). (SCRUM-10)
- **FR-1.6:** Backend integration for photo/file upload must be implemented. (SCRUM-9)
- **FR-1.7:** A message must be shown to the user after successful registration. (SCRUM-11)

FR-2: Automatic Face Recognition and Attendance

- **FR-2.1:** The system must have a camera integration system. (SCRUM-26, 4)
- **FR-2.2:** The system must be able to perform automatic face recognition and attendance tracking via camera. (SCRUM-2)
- **FR-2.3:** A face recognition and matching system must be established. (SCRUM-12)
- **FR-2.4:** The system must have an algorithm that compares the face from the camera with existing photos registered in the system. (SCRUM-13)
- **FR-2.5:** An accuracy threshold for matching must be determined. (SCRUM-14)
- **FR-2.6:** After a successful match, the attendance record must be created automatically. (SCRUM-15)
- **FR-2.7:** A real-time "Attendance taken" message must be displayed when attendance is recorded. (SCRUM-16)

FR-3: Reporting and Admin Panel (SCRUM-27)

- **FR-3.1:** There must be a reporting and management interface for attendance data. (SCRUM-3)
- **FR-3.2:** A page displaying daily attendance data must be created. (SCRUM-18)
- **FR-3.3:** The reporting page must have filtering and sorting features. (SCRUM-19)
- **FR-3.4:** Reports must be filterable by name. (SCRUM-20)
- **FR-3.5:** Reports must be sortable by department/class. (SCRUM-21)
- **FR-3.6:** The system must provide a warning for unrecognized students and allow manual attendance entry. (SCRUM-17)
- **FR-3.7:** The system must include a module for editing incorrect attendance records. (SCRUM-22)
- **FR-3.8:** Frontend integration for real-time updates in the admin panel must be implemented. (SCRUM-23)

2.2. Non-Functional Requirements (NFR)

NFR-1: Database and Infrastructure

- **NFR-1.1:** A database engine must be selected and installed. (SCRUM-34)
- **NFR-1.2:** Database setup and configuration must be performed. (SCRUM-33)
- **NFR-1.3:** A database connection configuration (connection string) must be defined. (SCRUM-35)
- **NFR-1.4:** An ORM (SQLAlchemy, Prisma, Django ORM, etc.) must be selected and developed. (SCRUM-38, 37)
- **NFR-1.5:** Database model classes (entities) must be created. (SCRUM-39)

NFR-2: Security

- **NFR-2.1:** Database security and access management must be ensured. (SCRUM-40)
- **NFR-2.2:** User roles must be defined and authorization implemented. (SCRUM-41)
- **NFR-2.3:** Encryption and access policies for sensitive data must be applied. (SCRUM-42)
- **NFR-2.4:** The system must have connection security (e.g., SSL). (SCRUM-43)
- **NFR-2.5:** Database access tests must be conducted. (SCRUM-36)

NFR-3: Performance and Maintenance

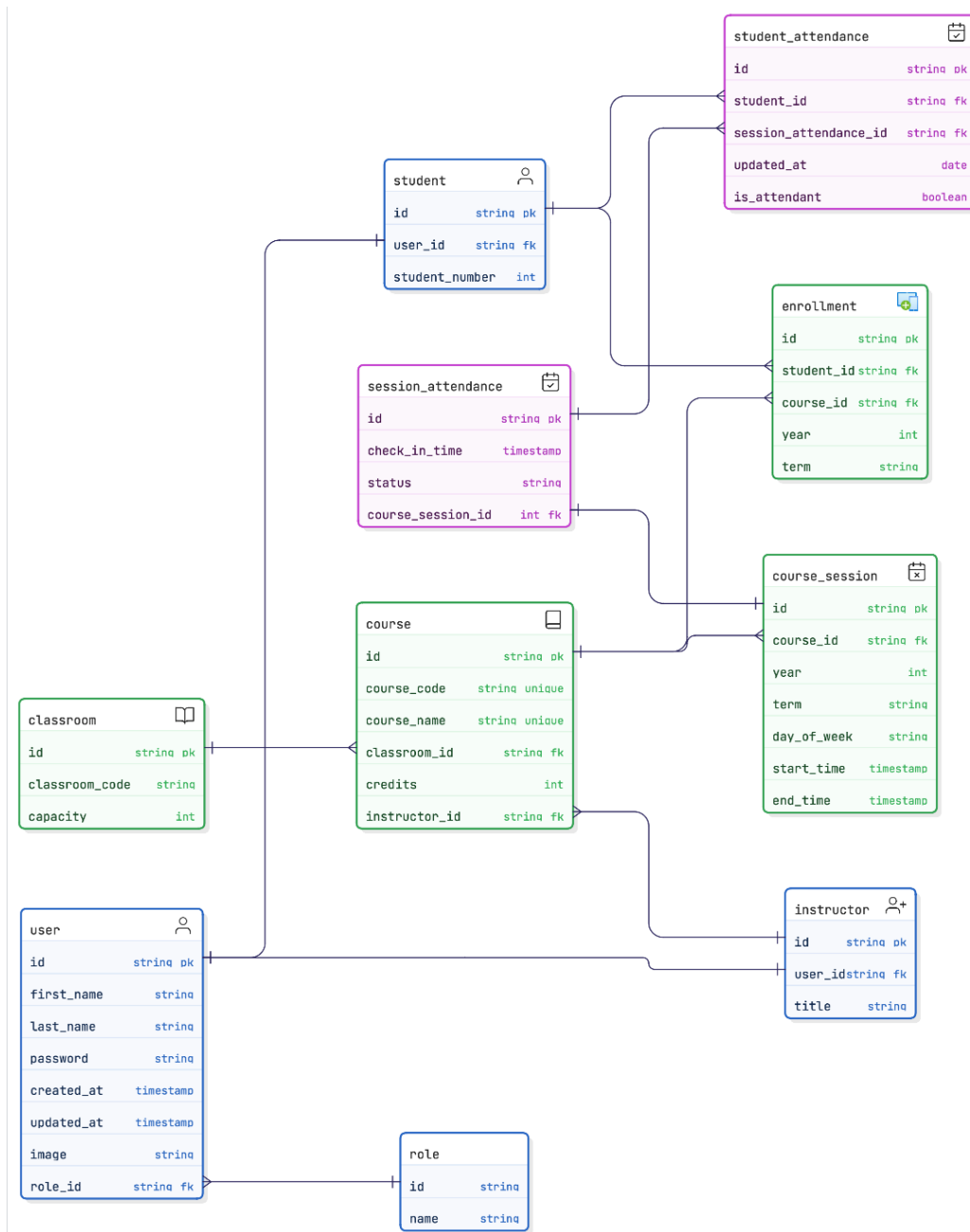
- **NFR-3.1:** Database backup and performance management must be planned. (SCRUM-44)
- **NFR-3.2:** An automatic backup plan must be created. (SCRUM-45)
- **NFR-3.3:** Query performance analyses must be conducted. (SCRUM-47)
- **NFR-3.4:** Necessary indexing strategies must be determined. (SCRUM-46)

NFR-4: Analysis and Documentation

- **NFR-4.1:** Requirements analysis and documentation for the project must be completed. (SCRUM-48, 49, 50, 51, 52, 53, 54)
- **NFR-4.2:** A requirements document (SRS) must be created. (SCRUM-55)

3. System Design

3.1 ERD (Entity Relationship Diagram)



3.2 Database Schema Design (PostgreSQL)

The database for the automated attendance system was implemented in **PostgreSQL** based on a Crow's Foot ERD. The schema is structured to ensure data integrity and enforce strict relationships between entities.

Users and Roles

- The User table is the core entity.
- Each user is linked to exactly one Student or Instructor, forming a one-to-one relationship through unique foreign keys.

Academic Structure

- Courses are managed via the Course, Classroom, and Instructor tables.
- Each course is assigned to a single instructor and a single classroom.
- Course codes and names are unique to prevent duplication.
- Weekly lectures are represented in the CourseSession table, which stores the day and time of each session.

Attendance Management

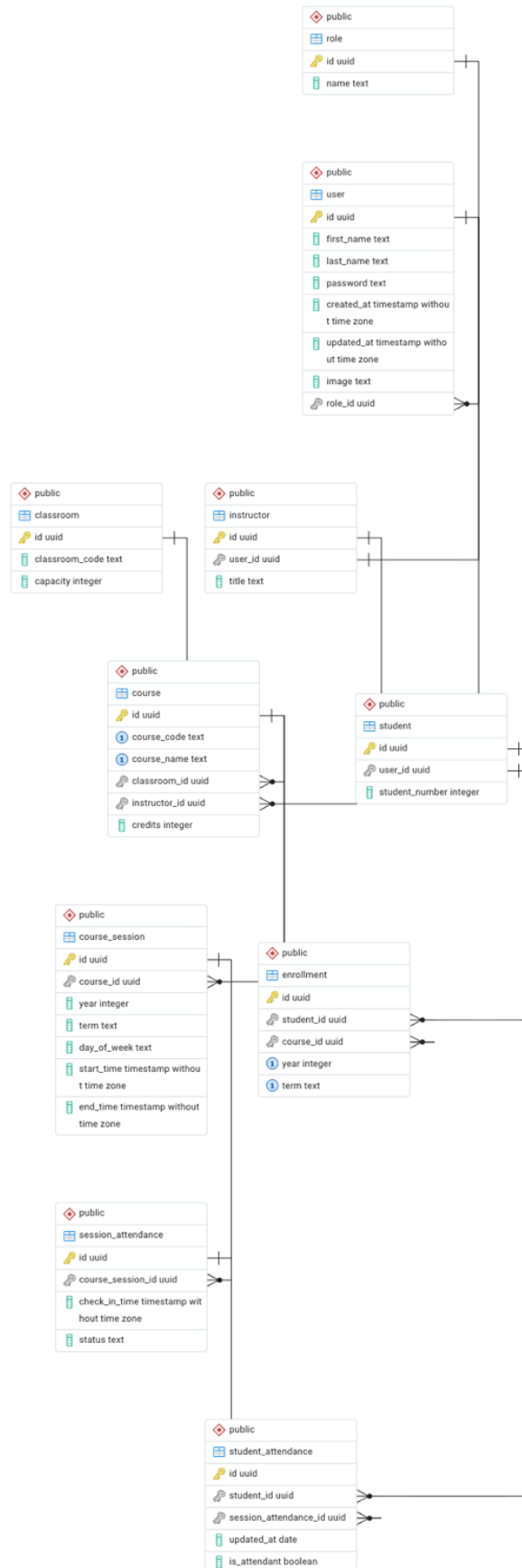
- Attendance is tracked through two tables: SessionAttendance for each session and StudentAttendance for individual student statuses.
- Unique constraints ensure that a student cannot have duplicate attendance entries for the same session.

Enrollment

- Students' course registrations are managed via the Enrollment table.
- The table enforces that a student cannot register for the same course more than once in the same term.

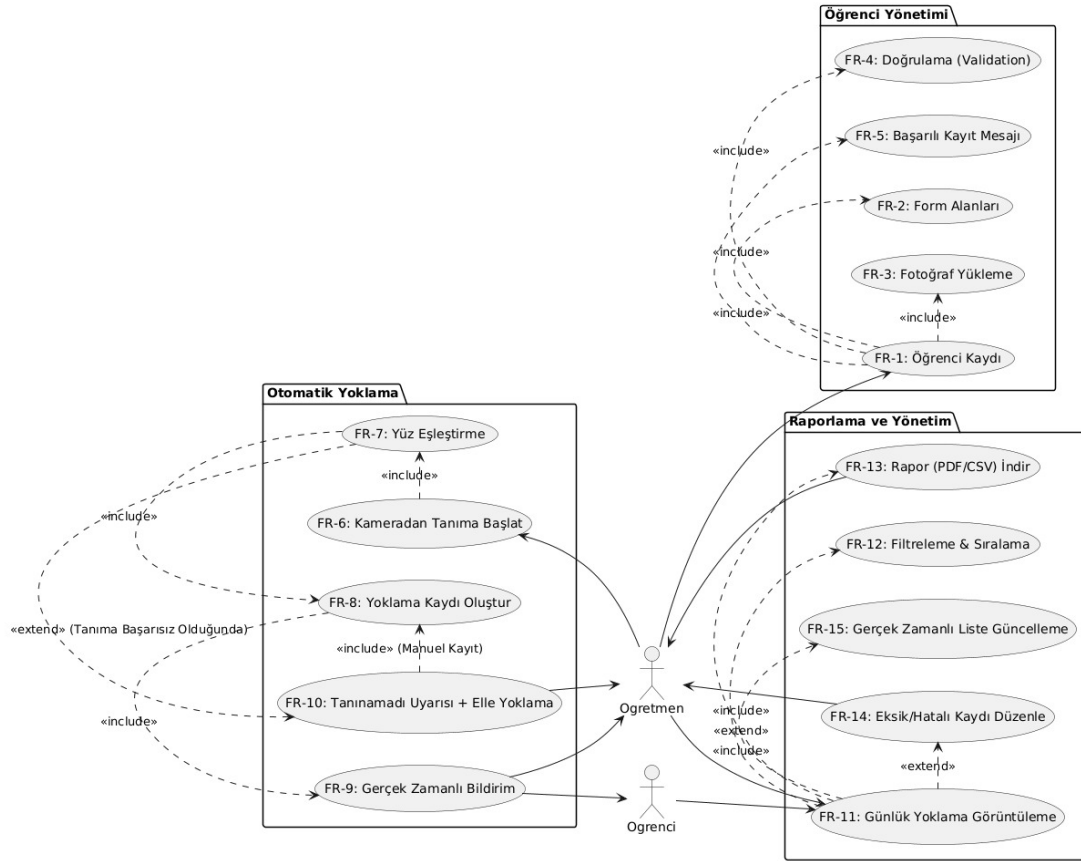
Data Integrity

- All primary keys, foreign keys, unique constraints, and required fields follow the ERD specifications.
- The schema was validated with SELECT queries to ensure that all relationships function correctly.



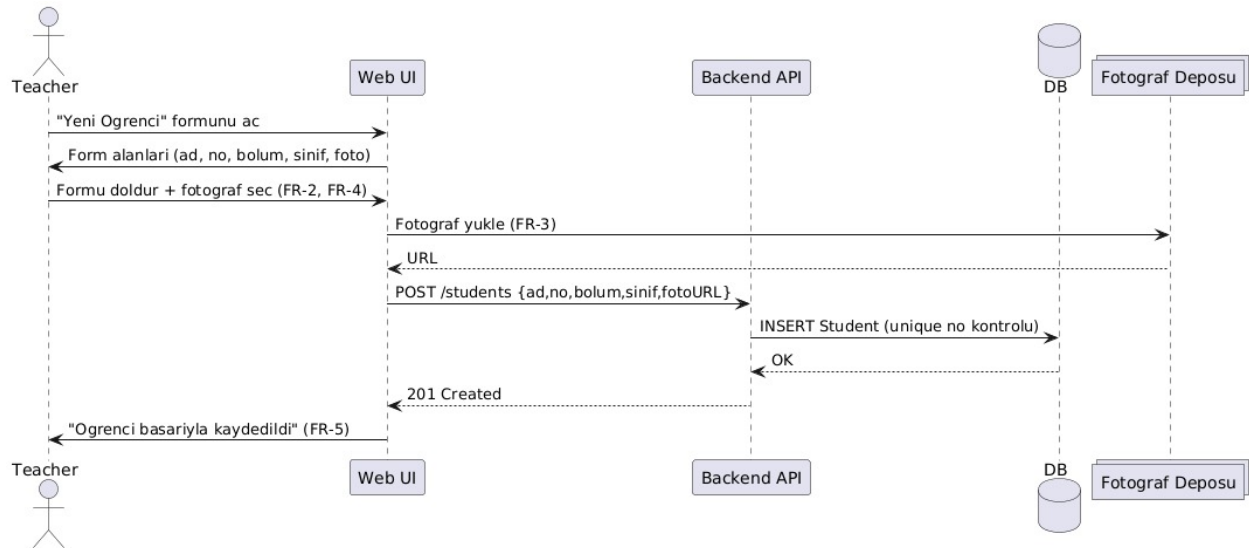
3.3 UML Diagrams

3.3.1 Use Case Diagram

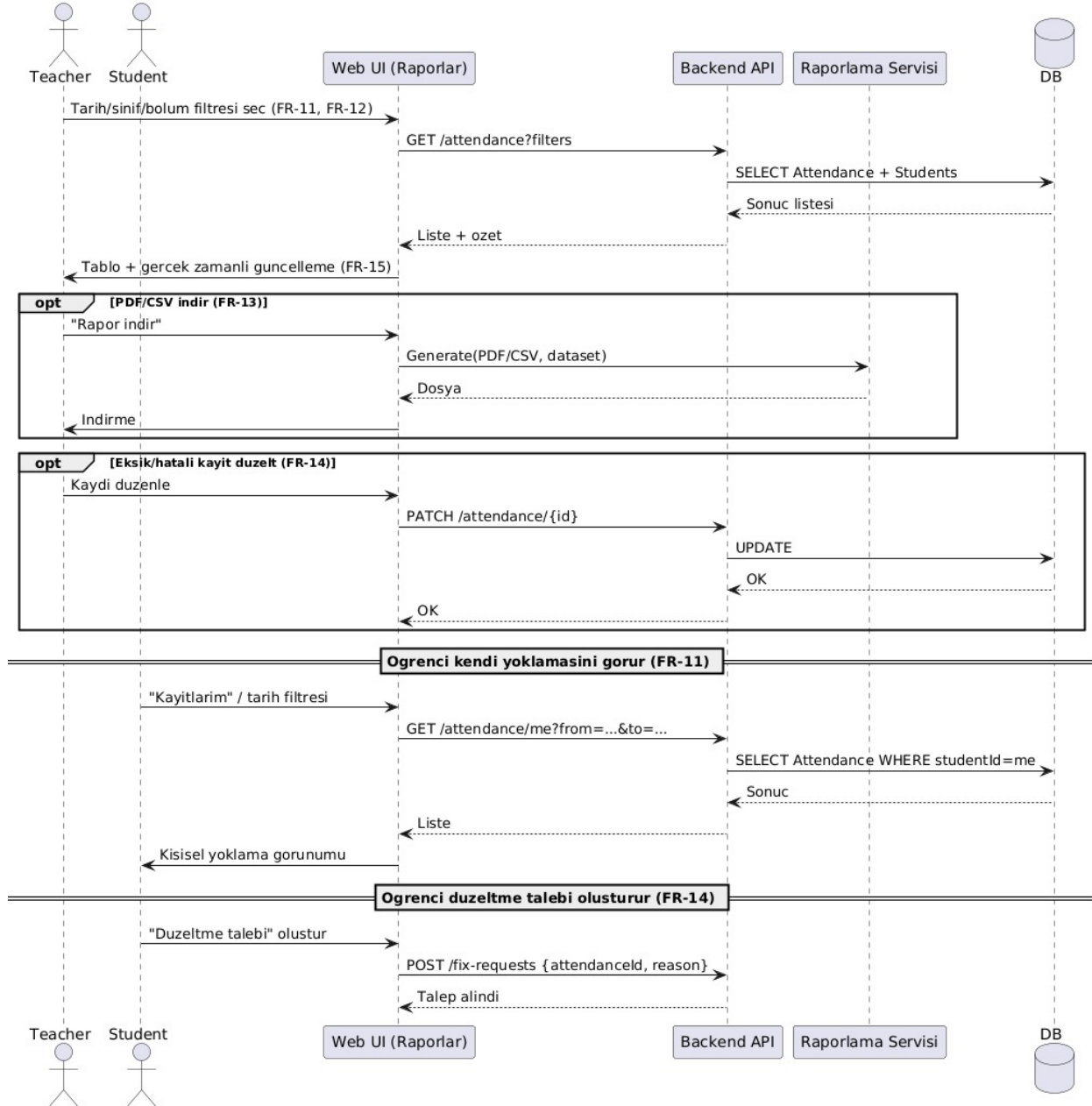


3.3.2 Sequence Diagrams

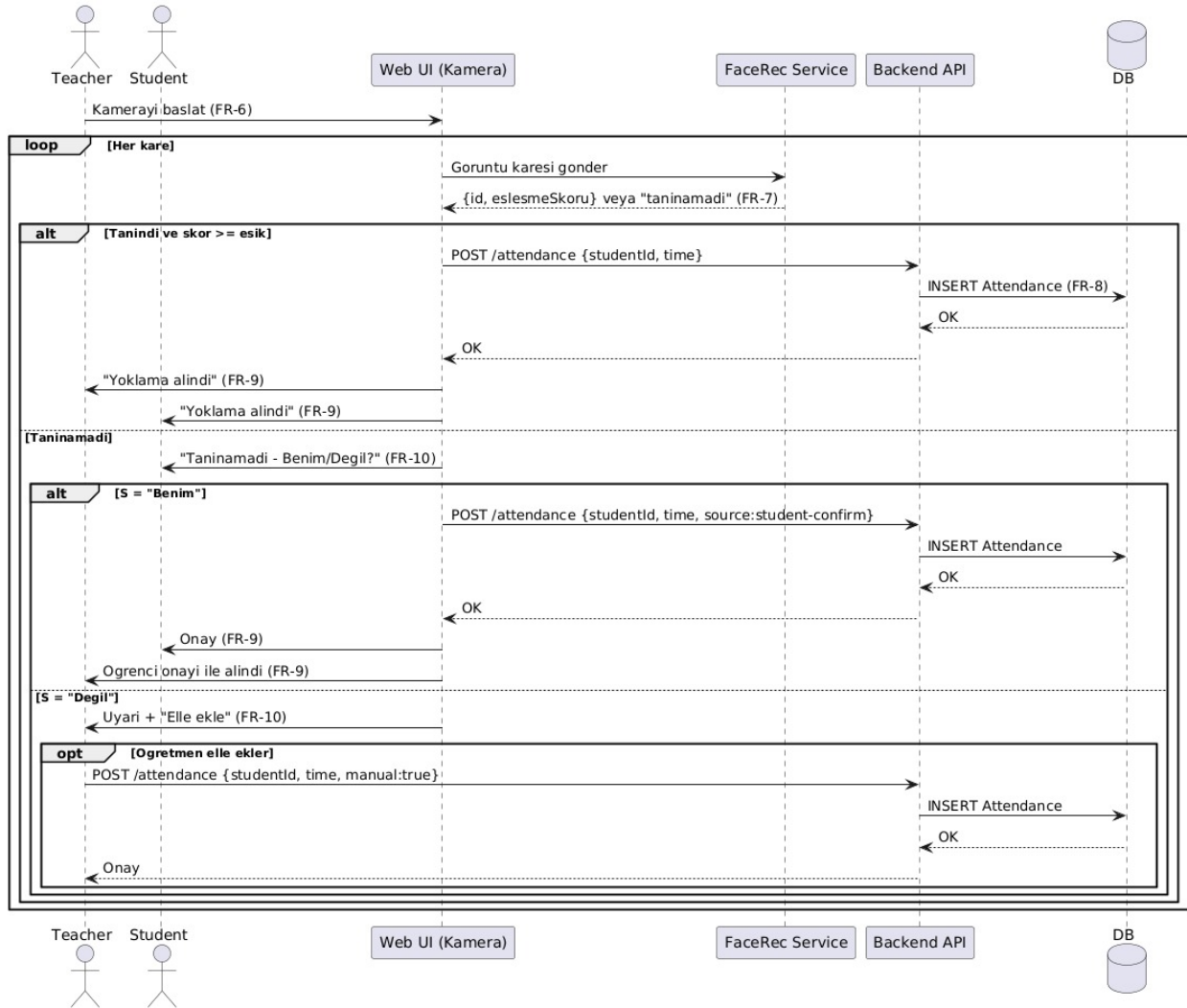
Student Registration



Automatic Attendance



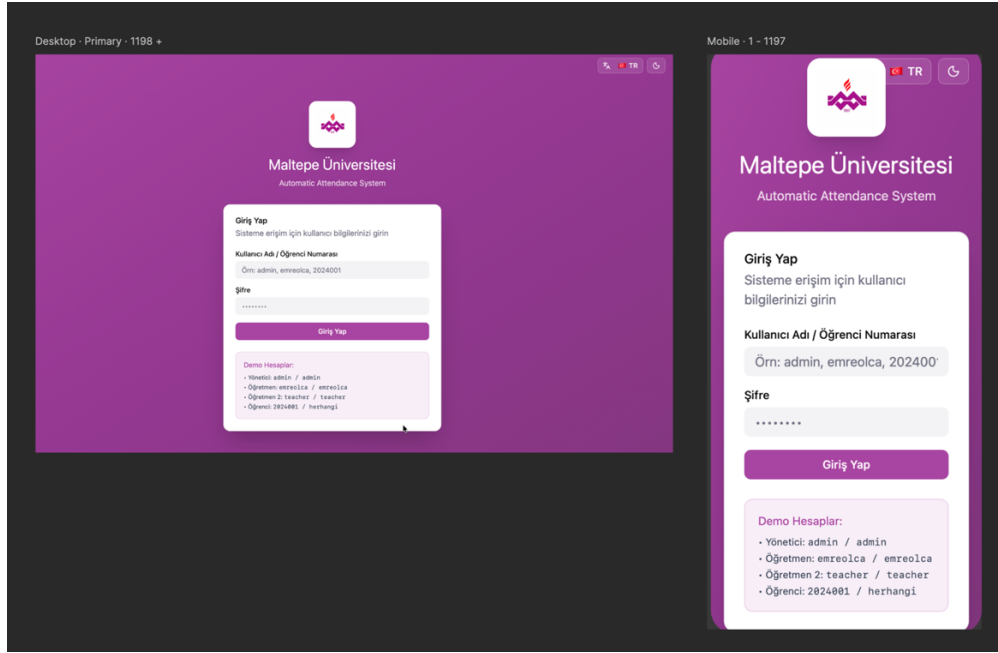
Attendance Reporting



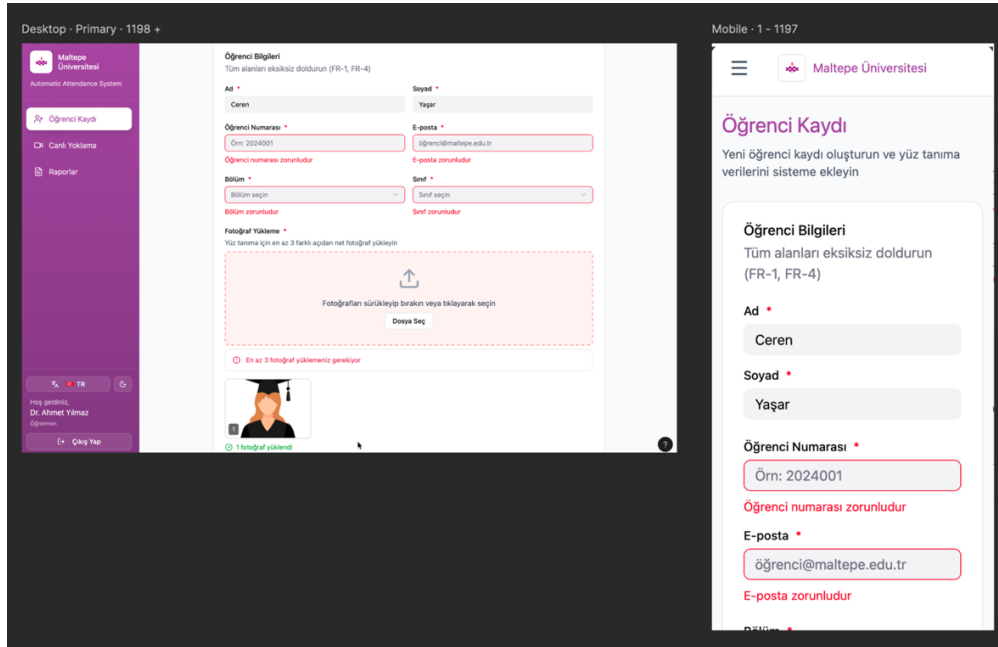
4. User Interface (UI) Design

The user interface design has been created in **Figma**, ensuring an interactive and visually clear layout.

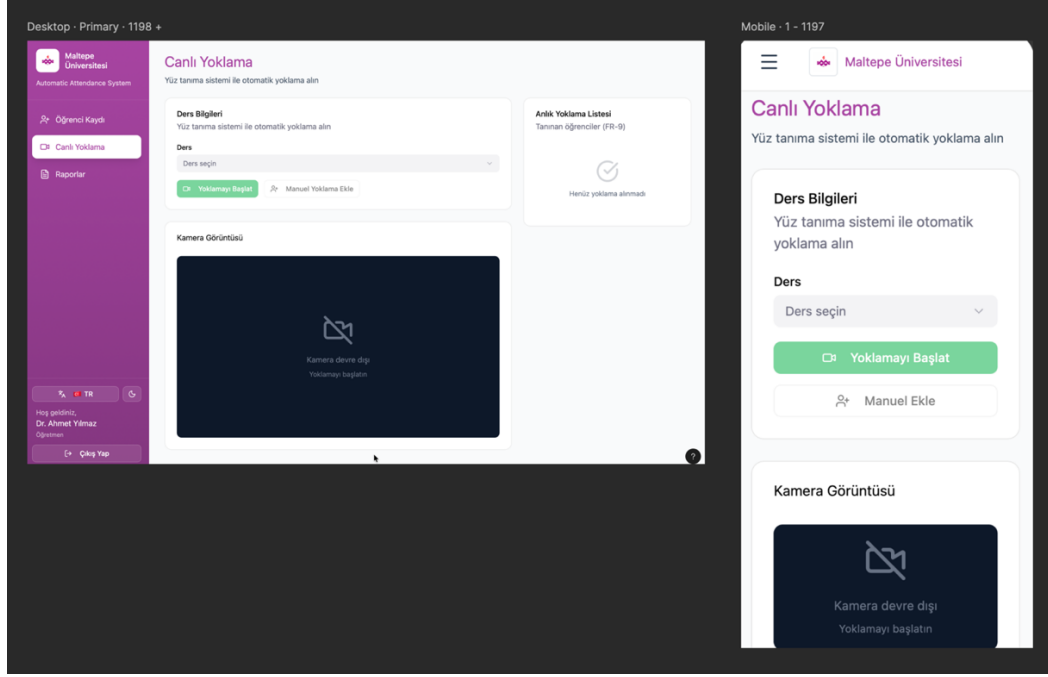
4.1 Log in Page



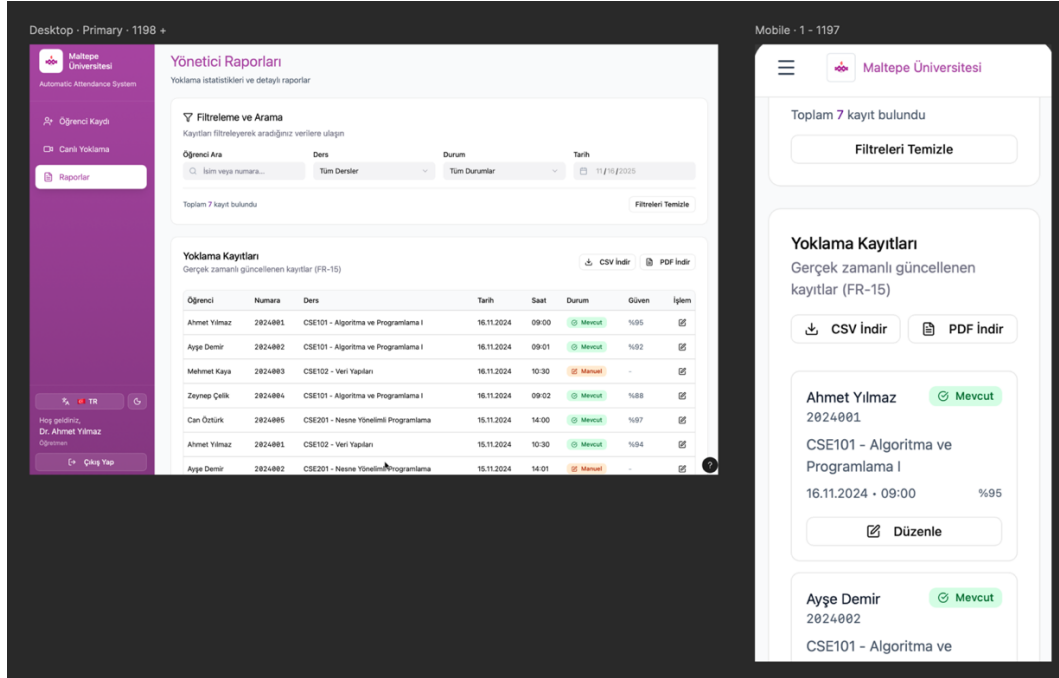
4.2 Student Registration Screen



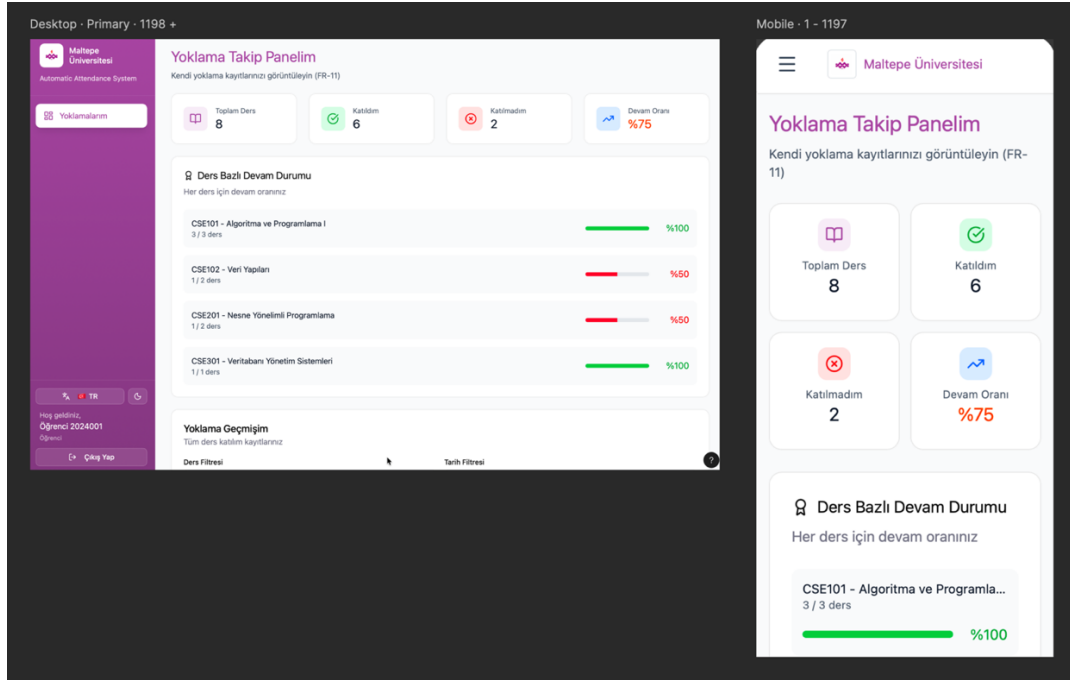
4.3 Automatic Attendance Session



4.4 Attendance Reports for Teacher



4.5 Student Reports Panel



5. Project Management & Methodology

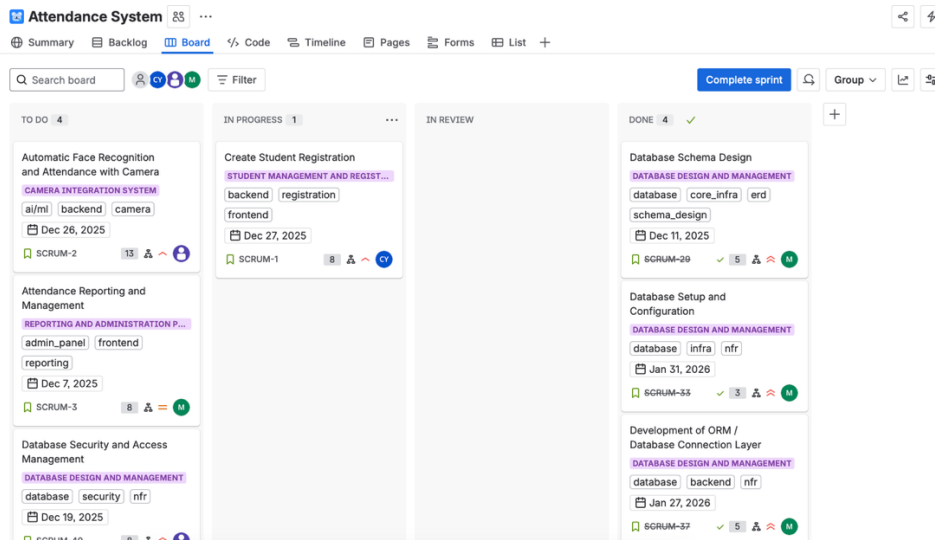
The project was executed using the **Agile Scrum Methodology**, utilizing professional management tools to track progress, assign tasks, and monitor sprint goals.

5.1. Project Tracking with Jira

Jira was the primary tool used for project management. It allowed the team to create a product backlog, define user stories (SCRUM items), and track the status of tasks through Kanban boards.

- **Task Management:** Each functional requirement was converted into a Jira task (e.g., SCRUM-25, SCRUM-26) and assigned to team members.
- **Sprint Planning:** Work was divided into sprints (Sprint 0 to Sprint 4), ensuring manageable deliverables for each phase.
- **Collaboration:** GitHub is used for code collaboration, pull requests, and code reviews to ensure software quality.
- **Validation & Testing:** Each sprint concludes with a testing phase where the implemented features are validated against the Functional Requirements (FR) and Non Functional Requirements (NFR).

Here is an image from the Jira board:



5.2 Sprint Planning

5.2.1 Sprint 0: Software Requirement and Analysis

- **Purpose:** To determine and document the project's scope, objectives, and functional/non-functional requirements.
- **Tasks:**
 - SCRUM-48: Software Requirement and Analysis
 - SCRUM-49: Gather Requirements from the Client
 - SCRUM-51: Schedule a meeting with the client and define the project scope...
 - SCRUM-52: Document needs and expectations
 - SCRUM-53: Prioritize requirements by categorizing them
 - SCRUM-50: Requirement Analysis and Documentation
 - SCRUM-54: Classify functional and non-functional requirements.
 - SCRUM-55: Create requirements document (SRS)

5.2.2 Sprint 1: Database Design and Management

- **Purpose:** To design, install, configure the database schema, and set up security and performance settings.
- **Tasks:**
 - SCRUM-28: Database Design and Management
 - SCRUM-29: Database Schema Design
 - SCRUM-30: Create ER Diagram
 - SCRUM-31: Determine Table and Column Types
 - SCRUM-32: Test Relationships (1-N, N-M)
 - SCRUM-33: Database Setup and Configuration
 - SCRUM-34: Select and Install Database Engine
 - SCRUM-35: Connection Configuration (connection string)
 - SCRUM-37: Develop ORM / Database Connection Layer
 - SCRUM-39: Create Model Classes

- SCRUM-38: Select ORM Structure (SQLAlchemy, Prisma, Django ORM)
- SCRUM-40: Database Security and Access Management
- SCRUM-43: Connection Security (SSL Environment)
- SCRUM-42: Apply Encryption and Access Policies
- SCRUM-41: Define User Roles
- SCRUM-44: Database Backup and Performance Management
- SCRUM-47: Analyze Query Performance
- SCRUM-46: Determine Indexing Strategies
- SCRUM-45: Create Automatic Backup Plan
- SCRUM-36: Perform Access Tests

5.2.3 Sprint 2: Student Management and Registration Module

- **Purpose:** To develop the necessary forms and backend integrations for administrators to create new student registrations.
- **Tasks:**
 - SCRUM-25: Student Management and Registration Module
 - SCRUM-1: Create Student Registration
 - SCRUM-5: Create student registration form
 - SCRUM-6: Add form fields (name, number, department, class, photo)
 - SCRUM-7: Add validation rules for form fields
 - SCRUM-8: Develop photo upload function
 - SCRUM-10: Store photos in the system
 - SCRUM-11: Display message after successful registration
 - SCRUM-9: File upload backend integration

5.2.4 Sprint 3: Camera Integration System

- **Purpose:** To integrate the camera system, set up face recognition and matching algorithms to automate attendance recording.
- **Tasks:**
 - SCRUM-26: Camera Integration System
 - SCRUM-2: Automatic Face Recognition and Attendance with Camera
 - SCRUM-4: Add camera integration
 - SCRUM-12: Set up face recognition and matching system
 - SCRUM-13: Comparison algorithm with existing photos
 - SCRUM-14: Determine the match accuracy threshold
 - SCRUM-15: Create attendance record automatically
 - SCRUM-16: Display real-time "Attendance taken" message

5.2.5 Sprint 4: Reporting and Admin Panel

- **Purpose:** To create an interface for administrators to view, filter, and manage the attendance records taken.
- **Tasks:**
 - SCRUM-27: Reporting and Admin Panel
 - SCRUM-3: Attendance Reporting and Management
 - SCRUM-18: Create daily attendance viewing page
 - SCRUM-19: Add filtering and sorting feature

- SCRUM-20: Filter by name
- SCRUM-21: Sort by department/class
- SCRUM-17: Warning for unrecognized student and manual attendance entry
- SCRUM-22: Add module to edit incorrect attendance
- SCRUM-23: Frontend integration for real-time updates

6. System Architecture & Technology Stack

The **Smart Attendance System** is developed as a modern web application, utilizing robust frameworks to ensure scalability and performance.



6.1 Technology Stack

6.1.1 Project Management: Jira by Atlassian

Jira The development process, including task tracking and sprint management, is conducted via Jira.

6.1.2 Database Management: PostgreSQL

PostgreSQL PostgreSQL is used as the relational database management system to securely store student data, attendance logs, and user credentials.

6.1.3 Frontend Framework: Why Next.js?

The user interface is developed using **Next.js**, chosen specifically for its **hybrid rendering capabilities (Server-Side Rendering and Client-Side Rendering)**. This architecture is critical for the "Smart Attendance System" for the following reasons:

Performance for Admin Dashboards: The system requires complex reporting panels and real-time data visualization. Next.js ensures fast initial page loads and efficient data fetching, reducing wait times for teachers accessing daily reports.

Scalability & Maintainability: Built on the React library, Next.js allows for a modular component-based structure. This makes it efficient to manage distinct interfaces like the *Camera Integration System* and *Student Registration Forms* independently without performance degradation.

Optimized User Experience: The framework handles routing and state management seamlessly, providing a responsive experience necessary for the instant "Attendance Recorded" notifications.

6.1.4 Backend & AI Engine: Why Python?

Core Application Logic & API: The core application logic and image processing services are powered by **Python**, utilizing **Flask** as the lightweight backend framework. This is the optimal choice for a face-recognition-based system due to Python's extensive AI ecosystem and Flask's flexibility in handling API requests.

Computer Vision Integration: Python provides seamless integration with essential libraries such as OpenCV and face_recognition. It handles the complex task of processing live video feeds, detecting faces, and converting them into encodings for comparison.

Efficient API Communication: **Flask** acts as the efficient bridge between the Next.js frontend and the database. By exposing RESTful API endpoints, it validates attendance inputs and manages the matching algorithms with high accuracy, ensuring smooth data flow.

Strategic Full-Stack Architecture: Coupling **Next.js** with a **Python/Flask** backend creates a robust, industry-standard architecture. This combination leverages the specific strengths of both technologies: Next.js ensures a high-performance, responsive user interface, while **Flask** manages the heavy computational load of AI processing on the server side. This separation of concerns ensures the system is both fast for the user and powerful in execution.

6.2 Version Control System

The project's source code is managed on **GitHub**, where a structured and functional branching strategy is applied to ensure clean collaboration and controlled development workflows.

- **main:** Represents the stable, production-ready branch. Only fully tested and approved features are merged into this branch to maintain high reliability.
- **frontend:** Dedicated to the development of the Next.js user interface. All UI components, layout implementations, and visual improvements are developed and tested here before integration.
- **backend:** Used for implementing server-side logic, API endpoints, authentication mechanisms, and database operations. This branch allows backend developers to work independently without affecting the UI development process.

7. System Implementation

1. Frontend Development (Next.js)

The client-side is built using **Next.js**, which ensures a highly responsive and dynamic user experience. Rather than serving static content, the frontend is designed to handle dynamic states:

- **Dynamic Rendering:** All dashboard elements are reactive. When a user logs in, the interface dynamically constructs the view based on their specific role (Student or Teacher).
- **API Consumption:** The frontend communicates with the backend via asynchronous API calls, sending user actions (like button clicks) and receiving real-time updates without refreshing the page.

2. Backend Architecture (Python Flask)

Python Flask serves as the central API gateway. It manages the application logic and acts as the secure intermediary between the client and the database.

- **RESTful Endpoints:** We implemented specific API routes (e.g., `/api/register-course`, `/api/get-attendance`) to handle incoming requests from the Next.js frontend.
- **Logic Processing:** Before any data reaches the database, Flask executes necessary validations—such as ensuring a student isn't already registered for a course before processing a new registration.

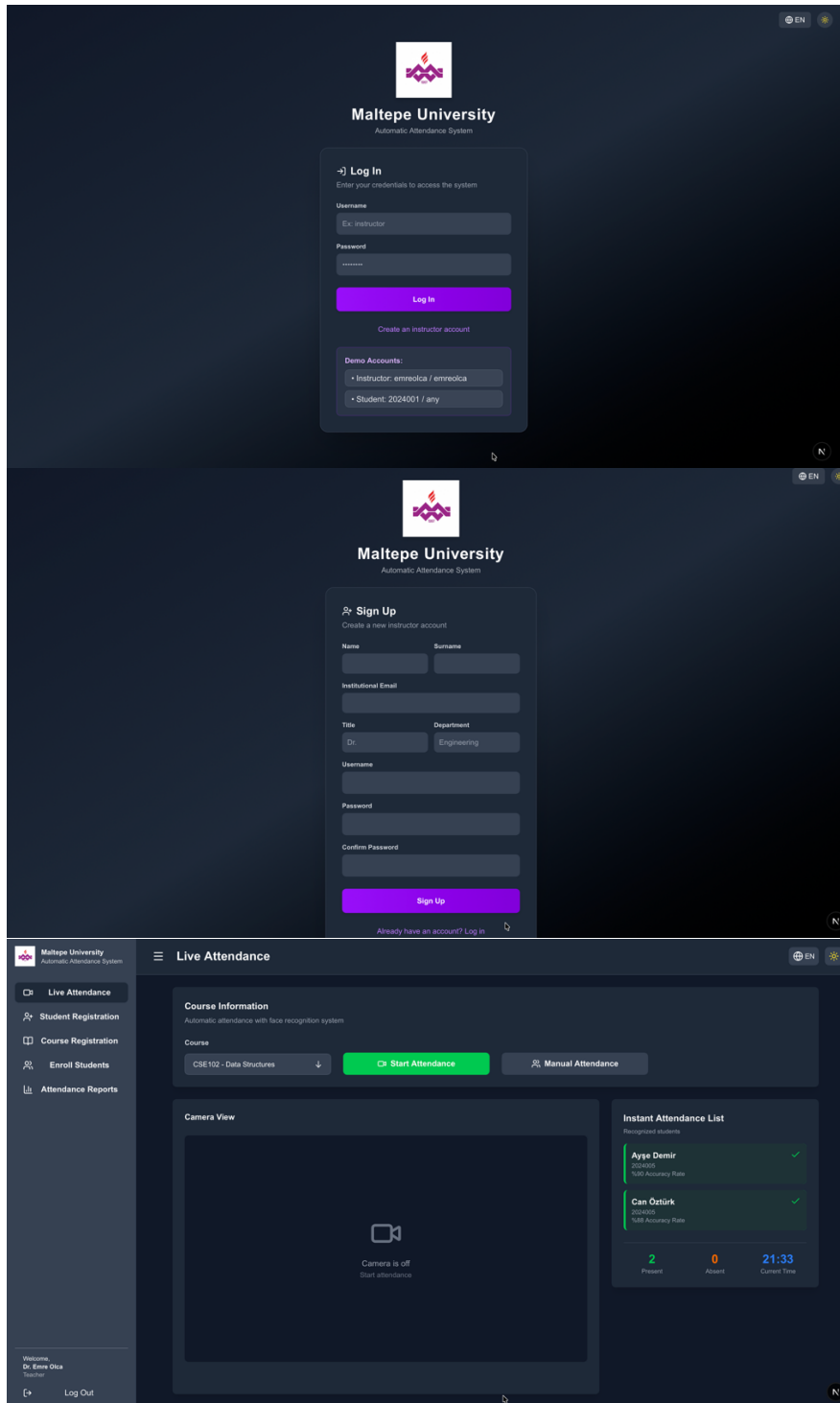
3. Database Integration & Data Flow (PostgreSQL)

The system relies on a robust **PostgreSQL** database to store and retrieve data persistently. The connection is managed securely through the Flask backend using object-relational mapping (ORM) techniques.

- **Dynamic Data Retrieval:** No hardcoded data is used. Information such as course lists, student IDs, and attendance records are fetched directly from PostgreSQL in real-time.
- **Course Registration Module:** A key feature of the implementation is the **Student Course Registration** system. When a student selects a course on the Next.js frontend:
 1. The request is sent to the Flask backend.
 2. Flask queries the PostgreSQL database to verify course availability.
 3. Upon validation, the database creates a relationship between the generic `Student` and `Course` tables.
 4. The updated course list is immediately reflected on the user's screen.

7.1 Teacher Interfaces

- Desktop Web Interfaces:



Maltepe University
Automatic Attendance System

Live Attendance

Student Registration

Course Registration

Enroll Students

Attendance Reports

Welcome,
Dr. Emre Olsa
Instructor

Log Out

Student Registration

Student Registration

Register a new student to the attendance system

Student Photo *

No photo

Click to upload photo
PNG, JPG up to 5MB

Student Number *

e.g., 2024001

Name *

First name

Surname *

Last name

Department *

Select department

Class *

Select class

Email

student@example.com

Phone

+90 555 123 4567

Register Student

Clear Form

Course Registration

Course Registration

Create and manage new courses

Course Code *

e.g., SE342

Course Name *

Software Engineering

Credits *

3

Instructor

Select instructor

Classroom

Select Classroom

Semester *

Select semester

Academic Year *

Select academic year

Course Description

Course description...

Register Course

Clear Form

Enroll Students to Course

Enroll Students to Course

Add registered students to courses

Select Course *

Select Course

Please select a course

22

Maltepe University
Automatic Attendance System

Live Attendance

Student Registration

Course Registration

Enroll Students

Attendance Reports

Welcome
Dr. Emre Oca
Instructor
Log Out

Attendance Reports

View, filter, and export attendance records

Total Records
9

Present
6

Absent
2

Late
1

Search
Q Search by student number, name...

Date
12/07/2025

Department
All Departments

Class
All Classes

Status
All Status

Export PDF

Export CSV

Student Number	Name	Department	Class	Date	Time	Status	Actions
2024001	Ahmet Yilmaz	CSE	Class 1	2024-11-20	09:15	Present	
2024002	Ayşe Demir	CSE	Class 1	2024-11-20	09:18	Present	
2024003	Mehmet Kaya	CSE	Class 1	2024-11-20	-	Absent	
2024004	Zeynep Şahin	CSE	Class 1	2024-11-20	09:22	Present	
2024005	Ali Çelik	CSE	Class 1	2024-11-20	09:35	Late	

EN

- Mobile Web Interfaces:

The image displays two mobile web interfaces for the Maltepe University Automatic Attendance System. Both screens feature a dark blue background with a white logo at the top center, consisting of a stylized 'M' with a red flame-like element. The top right of each screen has a globe icon and a sun icon.

Left Screen: Login Interface

- Header:** Maltepe University Automatic Attendance System
- Section:** →] Log In
- Text:** Enter your credentials to access the system
- Form Fields:**
 - Username:** Input field with placeholder text "Ex: instructor".
 - Password:** Input field with masked characters "*****".
- Button:** A large blue button labeled "Log In".
- Text:** Create an instructor account
- Demo Accounts:**
 - Instructor: emreolca / emreolca
 - Student: 2024001 / any
- Footer:** A circular icon with the letter 'N'.

Right Screen: Sign Up Interface

- Header:** Maltepe University Automatic Attendance System
- Section:** Sign Up
- Text:** Create a new instructor account
- Form Fields:**
 - Name:** Input field.
 - Surname:** Input field.
 - Institutional Email:** Input field.
 - Title:** Input field with placeholder text "Dr.".
 - Department:** Input field with placeholder text "Engineering".
 - Username:** Input field.
 - Password:** Input field.
 - Confirm Password:** Input field.
- Button:** A large blue button labeled "Sign Up".
- Text:** Already have an account? Log in
- Footer:** A circular icon with the letter 'N'.

≡ Live Attendance

Course Information

Automatic attendance with face recognition system

Course

CSE102 - Data Structures

Start Attendance

Manual Attendance

Camera View

Camera is off

Start attendance

Instant Attendance List

Recognized students

Ayşe Demir

2024005

%90 Accuracy Rate

Nan Öztürk

2024005

≡ Student Registration

Student Registration

Register a new student to the attendance system

Student Photo *

No photo

Click to upload photo

PNG, JPG up to 5MB

Student Number *

e.g., 2024001

Name *

First name

Surname *

Last name

Department *

Course Registration



Course Registration

Create and manage new courses

Course Code *

e.g., SE342

Course Name *

Software Engineering

Credits *

3

Instructor

Select Instructor

Classroom

Select Classroom

Semester *

Select semester

Academic Year *

Select academic year

Course Description

Course description

N

Enroll Students to Course



Enroll Students to Course

Add registered students to courses

Select Course *

Select Course

Please select a course

N

Attendance Reports

Attendance Reports

View, filter, and export attendance records

Total Records

9

Present

6

Absent

2

Late

1

Search

Q Search by student number, name...

Date

dd.mm.yyyy

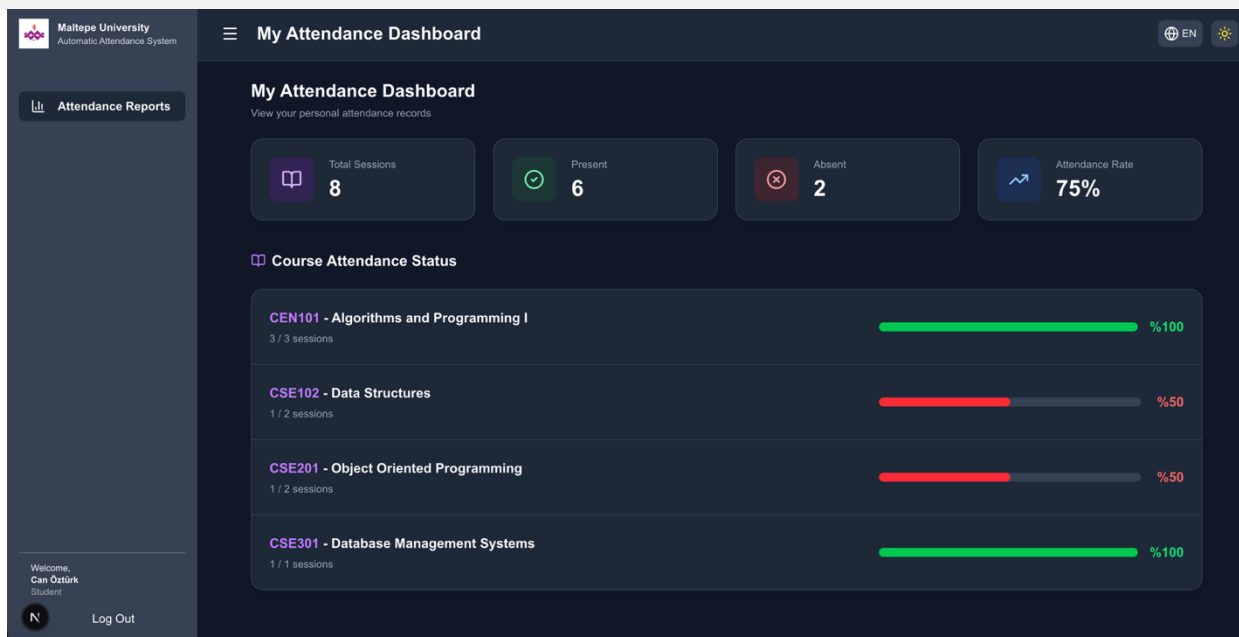
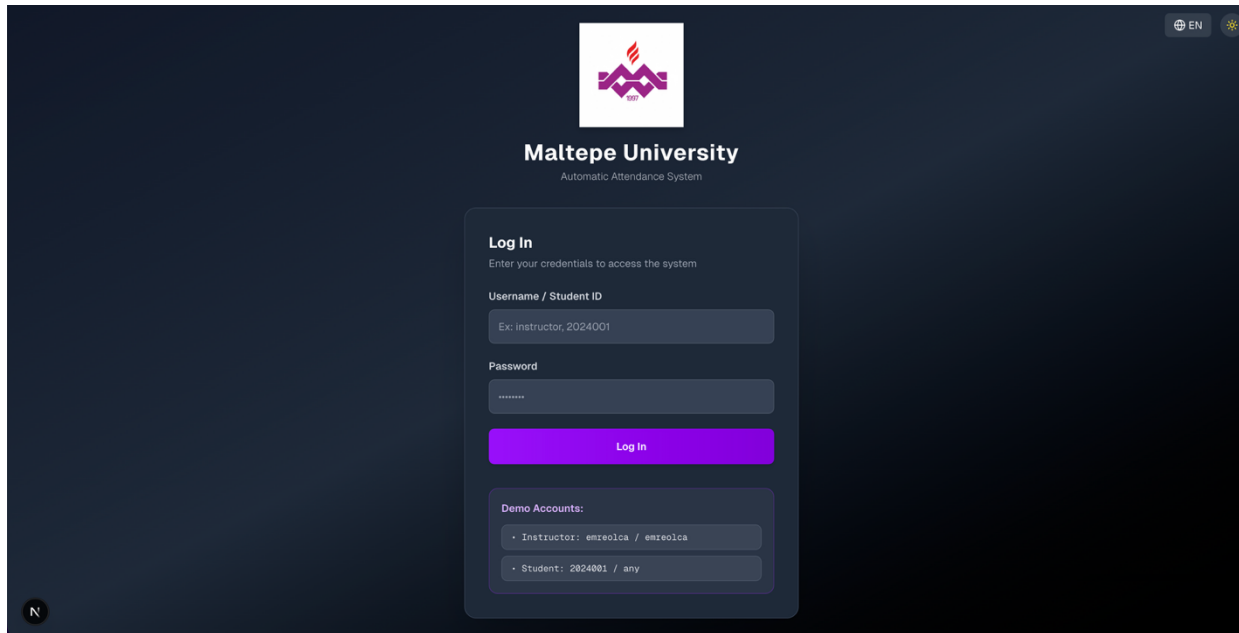
Department

N All Departments

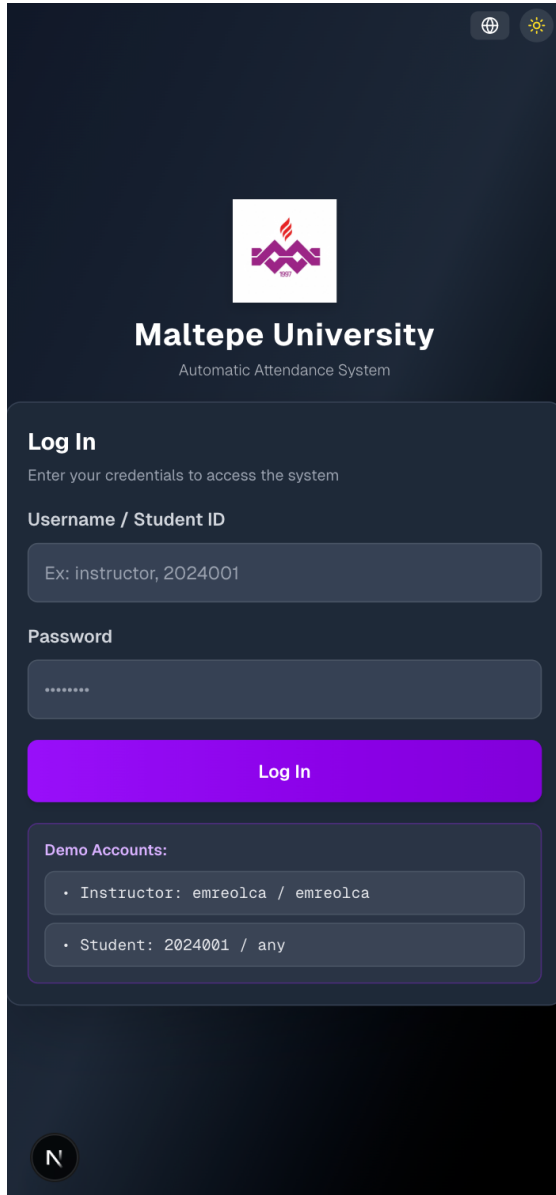
27

7.2 Student Interfaces

- Desktop Web Interfaces:



- Mobile Web Interfaces:



Maltepe University
Automatic Attendance System

Log In

Enter your credentials to access the system

Username / Student ID

Ex: instructor, 2024001

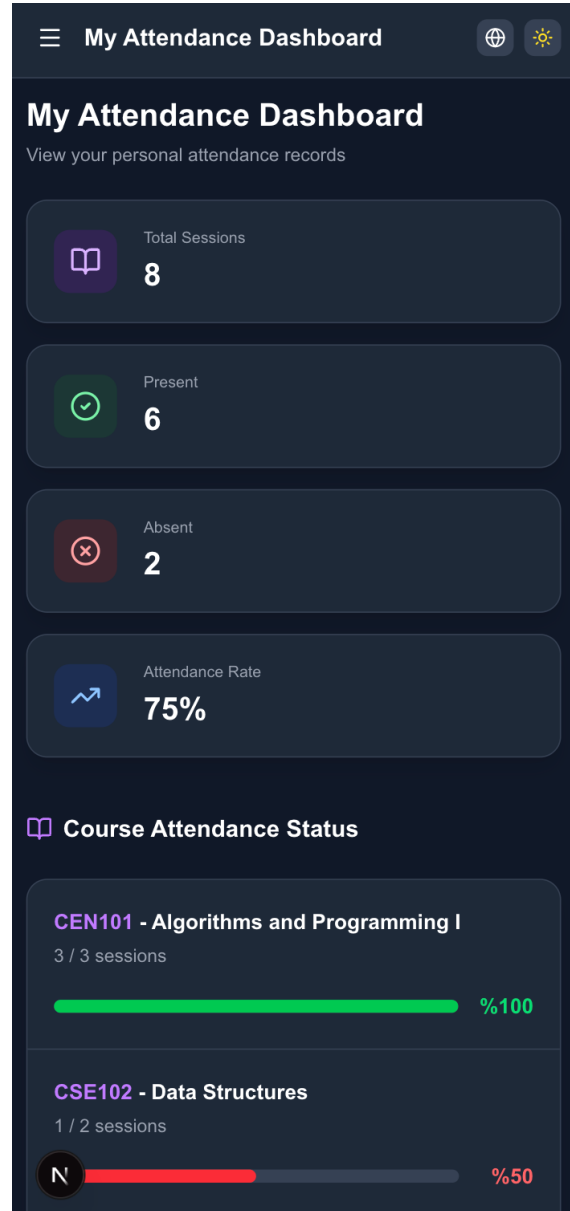
Password

Log In

Demo Accounts:

- Instructor: emreolca / emreolca
- Student: 2024001 / any

N



8. Test Development Process: Test Conditions, Test Cases and Test Procedures

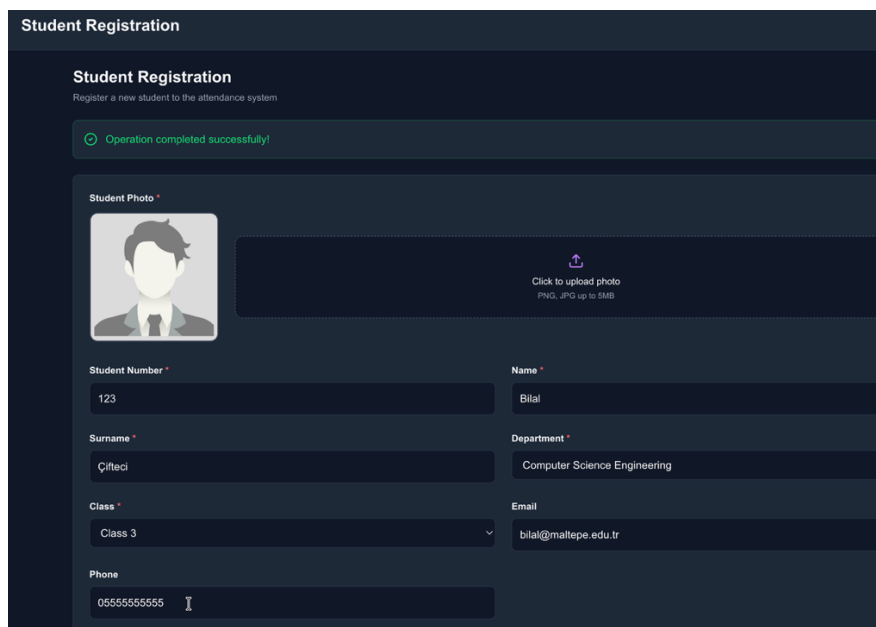
The test development process used in this study follows the standard approach of identifying test conditions, specifying test cases, and defining test procedures as presented in the course materials.

8.1 Test Cases for Student Management Page

Source Files: frontend/app/student-registration/page.tsx, backend/routes.py

Test Case 1 - Test Condition: Registration should be successful when all required fields are entered correctly and a photo is uploaded.

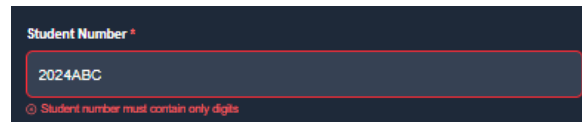
- **A. Test Cases:**
 - **Precondition:** User is on the "Student Registration" page.
 - **Inputs (Valid):** Number: "2024001", Name: "Ceren", Surname: "Yaşar", Department: "Computer Science Engineering", Class: "3", Email: "ceren@test.com", Photo: "profile.jpg" -> **Expected Result:** "Student registered successfully!" message is displayed.
- **B. Test Procedure:**
 1. Fill in all the fields in the form with valid data.
 2. Select a valid image file from the "Click to upload photo" area.
 3. Click the "Register Student" button.
 4. Verify the green success message is displayed.



The screenshot displays the 'Student Registration' form. At the top, a green message bar states 'Operation completed successfully!'. Below this, there is a 'Student Photo' section with a placeholder image and a 'Click to upload photo' button. The form fields are organized into two columns. The left column contains 'Student Number' (123), 'Surname' (Çiftci), 'Class' (Class 3), and 'Phone' (05555555555). The right column contains 'Name' (Bilal), 'Department' (Computer Science Engineering), and 'Email' (bilal@maltepe.edu.tr). All fields are filled with valid data.

Test Case 2 - Test Condition: Student number must consist of digits only.

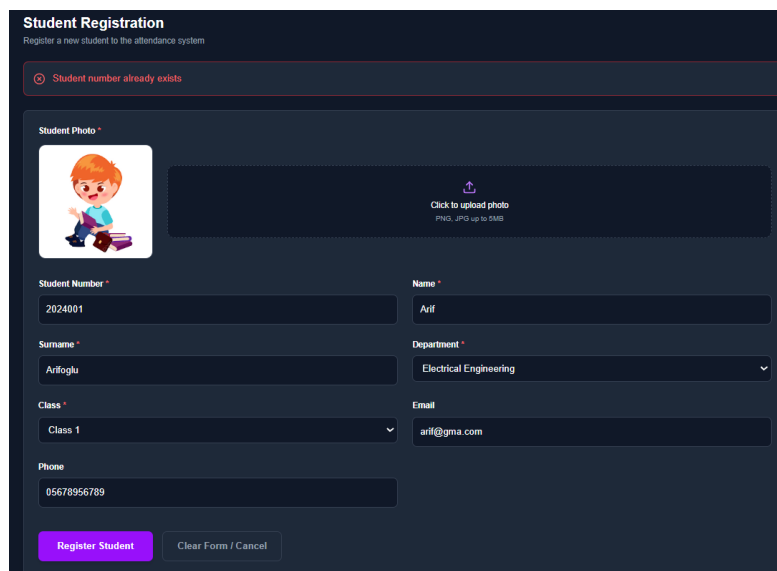
- **A. Test Cases:**
 - **Inputs (Invalid):** Student Number: "2024ABC" -> **Expected Result:** "Student number must contain only digits" error message.
- **B. Test Procedure:**
 1. Enter a value containing letters into the Student Number field.
 2. Click the Register button.
 3. Check for the red error message below the field.



A screenshot of a web form titled "Student Number *". The input field contains the text "2024ABC". Below the field, a red error message is displayed: "ⓘ Student number must contain only digits".

Test Case 3 - Test Condition: A second registration cannot be made with the same student number (Unique Constraint).

- **A. Test Cases:**
 - **Inputs (Invalid):** Student Number: "2024001" (Already registered) -> **Expected Result:** "Student number already exists" error message.
- **B. Test Procedure:**
 1. Enter a student number that is already registered in the system.
 2. Submit the form.
 3. Verify the conflict error (409 Conflict) returned from the backend.



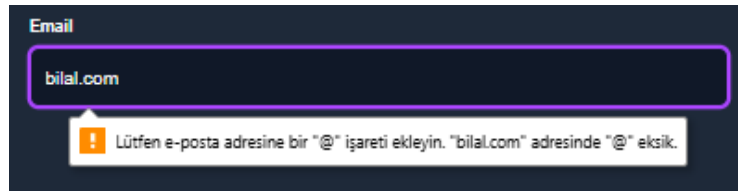
A screenshot of the "Student Registration" form. At the top, a red error message states "ⓘ Student number already exists". The form includes fields for "Student Photo" (with a placeholder image of a student), "Student Number" (containing "2024001"), "Name" (containing "Arif"), "Surname" (containing "Arifoglu"), "Department" (a dropdown menu showing "Electrical Engineering"), "Class" (a dropdown menu showing "Class 1"), "Email" (containing "arif@gmail.com"), and "Phone" (containing "05678956789"). At the bottom, there are two buttons: "Register Student" (in purple) and "Clear Form / Cancel" (in grey).

Test Case 4 - Test Condition: Email address must be in a valid format.

- **A. Test Cases:**
 - **Inputs (Invalid):** Email: "cerenyasar" (No domain) -> **Expected Result:** "Please enter a valid email address" error message.

- **B. Test Procedure:**

1. Enter text without an '@' symbol in the Email field.
2. Click the Register button.
3. Observe the validation error.



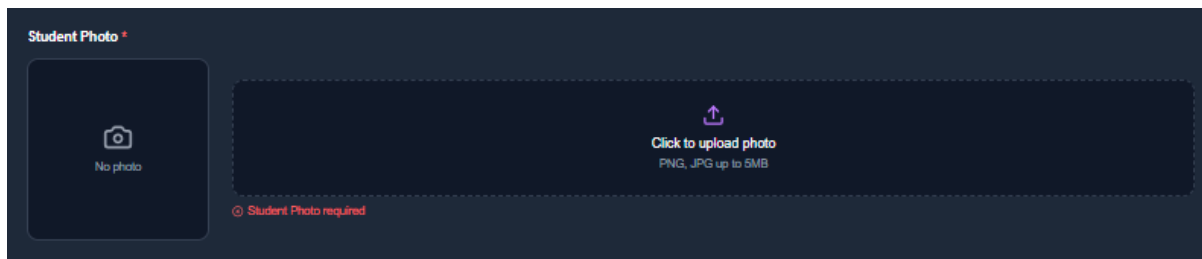
Test Case 5 - Test Condition: Registration cannot be performed without uploading a photo.

- **A. Test Cases:**

- **Inputs (Invalid):** Photo: [Empty] -> **Expected Result:** "Student Photo Required" warning.

- **B. Test Procedure:**

1. Fill in the text fields but do not select a photo.
2. Click the Register button.
3. Verify that an error warning appears in the photo area.



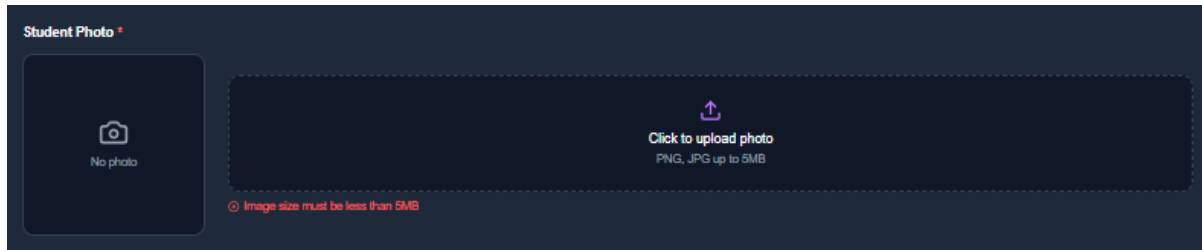
Test Case 6 - Test Condition: Uploaded file size must not exceed the 5MB limit.

- **A. Test Cases:**

- **Inputs (Invalid):** Photo: "photo.jpg" (6MB) -> **Expected Result:** "Image size must be less than 5MB" error message.

- **B. Test Procedure:**

1. Select a file larger than 5MB from your computer.
2. Verify that the error message appears immediately upon file selection.

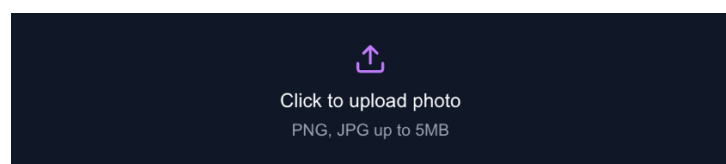


Test Case 7 - Test Condition: Phone number field is optional and can be left blank.

- **A. Test Cases:**
 - **Inputs (Valid):** Phone: [Empty] -> **Expected Result:** Registration is successful.
- **B. Test Procedure:**
 1. Fill in the required fields, leave the phone blank.
 2. Click the Register button.
 3. Verify that the process is successful.

Test Case 8 - Test Condition: Only image file formats (jpg, png) should be accepted.

- **A. Test Cases:**
 - **Inputs (Invalid):** File: "photo.pdf" -> **Expected Result:** "Please select an image file" error message.
- **B. Test Procedure:**
 1. Try to select a PDF or Word document in the photo upload area.
 2. Verify that the system rejects the file type.



8.2 Test Cases for Course Registration and Course Enrollment

Source Files: backend/models.py, backend/routes.py, frontend/app/course-enrollment/page.tsx

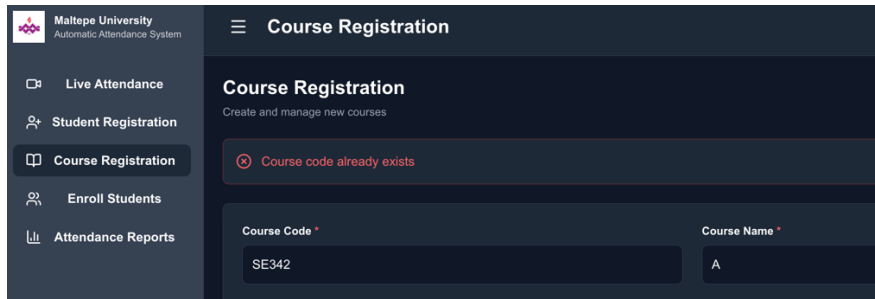
Test Case 1 - Test Condition: A new course should be created with valid code, name, and credit information.

- **A. Test Cases:**
 - **Precondition:** User is on the Course Creation screen.
 - **Inputs (Valid):** Code: "SE342", Name: "Software Validation", Credits: 6 -
> **Expected Result:** Course is created successfully (HTTP 201).
- **B. Test Procedure:**
 1. Fill in the Course Code, Name, and Credits fields.
 2. Click the "Create" button.
 3. See the new course in the list.

The image contains two screenshots of a web application interface. The left screenshot shows a 'Kurs Kayıt' (Course Registration) form. It has fields for 'Kurs Kodu' (Course Code) with the value 'SE342', 'Kurs Adı' (Course Name) with the value 'Software Validation', 'Credits' with the value '6', 'Classroom' with the value '206 (Cap 50)', and 'Akademik Yıl' (Academic Year) with the value '2025-2026'. There is a 'Kurs Açıklaması' (Course Description) field with the value 'Software Validation'. At the bottom, there is a 'Kursu Kaydet' (Save Course) button and a 'Formu Temizle' (Clear Form) button. The right screenshot shows a 'Kursa Öğrenci Ekle' (Add Student to Course) screen. It has a 'Kurs Seçin' (Select Course) dropdown menu. Below the dropdown, there is a list of courses: 'SE342 - Data Structures (Credits: 3)', 'GEN300 - OOP (Credits: 4)', 'GEN101 - Algorithm Programming (Credits: 3)', 'MATH102 - CALCULUS 2 (Credits: 5)', and 'SE3421 - Software Validation (Credits: 6)'. The 'SE3421 - Software Validation (Credits: 6)' course is highlighted in blue.

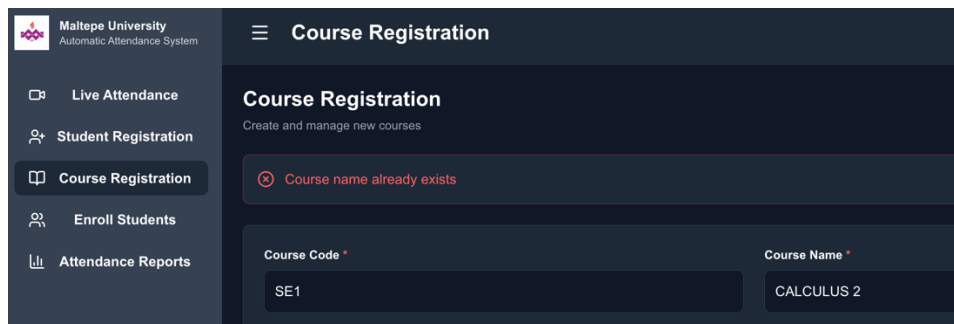
Test Case 2 - Test Condition: A second course cannot be added with the same course code (Unique Code).

- **A. Test Cases:**
 - **Inputs (Invalid):** Code: "SE342" (Existing) -> **Expected Result:** System gives an error (Database Integrity Error).
- **B. Test Procedure:**
 1. Enter an existing course code again.
 2. Try to save.
 3. Verify the error message.



Test Case 3 - Test Condition: A course with an existing name cannot be added even with a different code (Unique Name).

- **A. Test Cases:**
 - **Inputs (Invalid):** Code: "MATH101", Name: "Calculus" (Name exists) -
> **Expected Result:** Name conflict error.
- **B. Test Procedure:**
 1. Enter a different code but type a course name that already exists.
 2. Click the Save button.
 3. Observe the conflict warning.



Test Case 4 - Test Condition: Course credits field cannot be left blank.

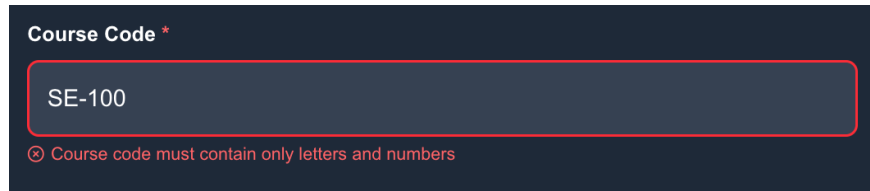
- **A. Test Cases:**
 - **Inputs (Invalid):** Credits: [Empty] -> **Expected Result:** "Missing required fields" error.
- **B. Test Procedure:**
 1. Leave the Credits field blank.
 2. Submit the form.
 3. Check for the required field warning.

Test Case 5 – Test Condition: A course **cannot** be created without selecting an instructor (Mandatory field).

- **A. Test Cases:**
 - **Inputs:** Instructor: [Not Selected] -> **Expected Result:** The system prevents the course creation and displays an error message (e.g., "Please select an instructor").
- **B. Test Procedure:**
 1. Enter course details, but **skip** the instructor dropdown menu (leave it empty).
 2. Click the **Save** button.
 3. Verify that the course is **not** created and a validation error (e.g., "Instructor field is required") is displayed.

Test Case 6 – Test Condition: The Course Code must only contain alphanumeric characters (letters and numbers); special characters or spaces are not allowed.

- **A. Test Cases:**
 - **Inputs (Invalid):**
 - Course Code: "SE-100" (Contains hyphen) OR "SE 100" (Contains space)
 - Other fields: Valid
 - **Expected Result:** The system blocks the submission and displays a validation error: *"Course code must contain only letters and numbers"*.
- **B. Test Procedure:**
 1. Navigate to the Course Registration page.
 2. Enter "SE-342" (or any code with a symbol/space) into the **Course Code** field.
 3. Fill in the rest of the form with valid data.
 4. Click the **Register Course** button.
 5. Verify that the form is **not** submitted and the format error message is displayed.



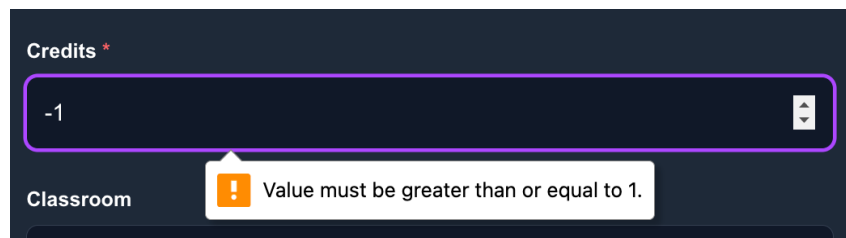
Course Code *

SE-100

⊗ Course code must contain only letters and numbers

Test Case 7 - Test Condition: Course credits must be numeric and positive integer.

- **A. Test Cases:**
 - **Inputs (Invalid):** Credits: "Five" -> **Expected Result:** Input is rejected or an error is shown.
- **B. Test Procedure:**
 1. Try to enter text into the Credits field.
 2. Verify that the system prevents non-numeric input.



Credits *

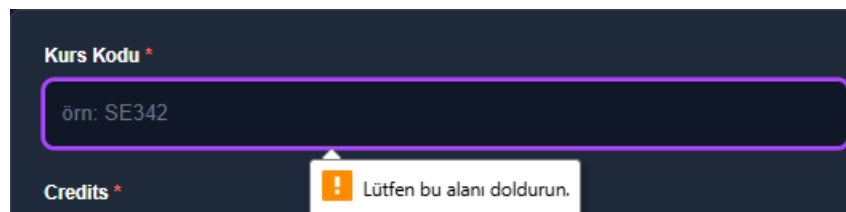
-1

Classroom

! Value must be greater than or equal to 1.

Test Case 8 - Test Condition: Course code cannot be left blank.

- **A. Test Cases:**
 - **Inputs (Invalid):** Code: [Empty] -> **Expected Result:** Required field warning.
- **B. Test Procedure:**
 1. Clear or leave the Code field blank.
 2. Click the Save button.
 3. Verify that the operation does not proceed.



Kurs Kodu *

örn: SE342

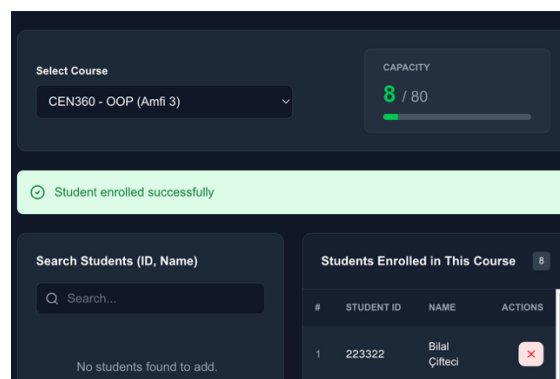
Credits *

! Lütfen bu alanı doldurun.

Test Case 9 – Test Condition: Successfully enroll a student into a course.

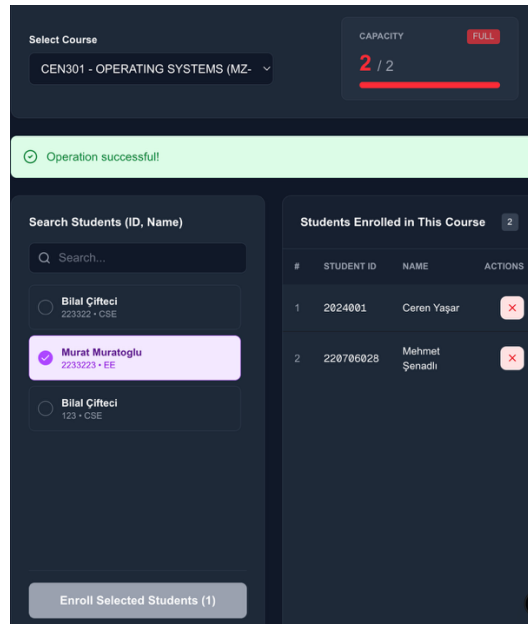
- **A. Test Cases:**
 - **Precondition:** A course "SE342" exists with capacity 50. Student "2023001" exists.
 - **Inputs (Valid):**

- Course: "SE342"
 - Student Number: "2023001"
 - **Expected Result:** System displays *"Student enrolled successfully!"*, the student appears in the "Enrolled Students" list, and the capacity counter increases by 1 (e.g., 1/50).
- **B. Test Procedure:**
 1. Navigate to "Enroll Students" page.
 2. Select "SE342" from the dropdown.
 3. Enter "2023001" into the search/input box.
 4. Click the "Enroll" button.
 5. Verify the success message and check the updated list.



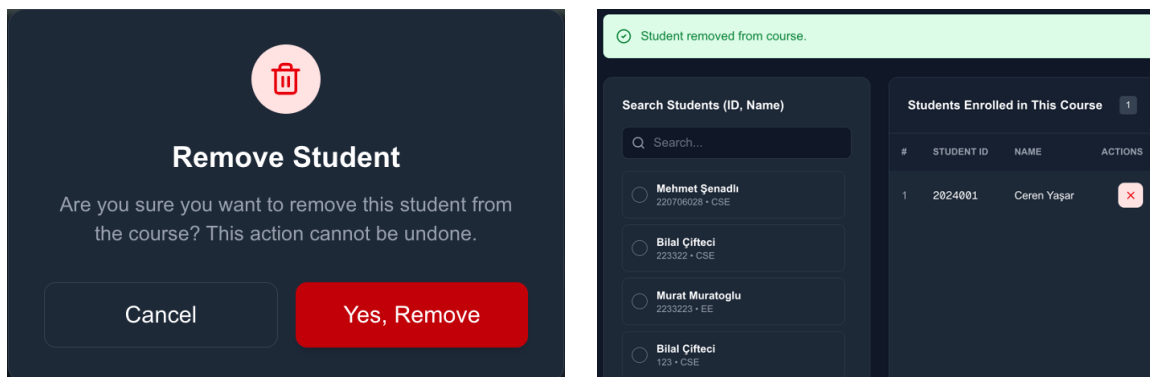
Test Case 10 – Test Condition: Prevent enrollment if the course capacity is full.

- **A. Test Cases:**
 - **Precondition:** A course "CEN301" exists with **Capacity: 2** and **Enrolled: 1** (Full).
 - **Inputs (Invalid):**
 - Course: "LAB101"
 - Student Number: "2023002" (New Student)
 - **Expected Result:** System blocks the action and displays an error message: *"Classroom is full! Capacity: 1"*. The student is **not** added.
- **B. Test Procedure:**
 1. Select the full course "CEN301".
 2. Observe the "FULL" badge on the capacity indicator.
 3. Try to enroll a new student.
 4. Verify that the error message appears and the count remains 2/2.



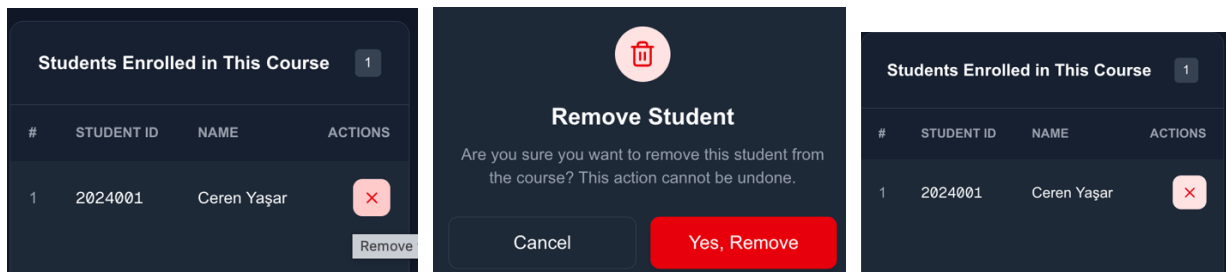
Test Case 11 – Test Condition: Remove a student from a course (Unenroll).

- **A. Test Cases:**
 - **Precondition:** Student "2023001" is enrolled in "SE342".
 - **Action:** Click the "X" (Remove) button next to the student's name in the list.
 - **Expected Result:** A confirmation modal appears. Upon clicking "Yes/Delete", the student is removed from the list, and the capacity counter decreases by 1.
- **B. Test Procedure:**
 1. Select "SE342".
 2. Find student "2023001" in the list.
 3. Click the Remove (X) button.
 4. Confirm the action in the pop-up modal.
 5. Verify the student is gone from the list and the count decreased.



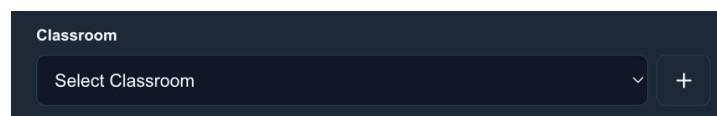
Test Case 12 – Test Condition: Cancel student removal action.

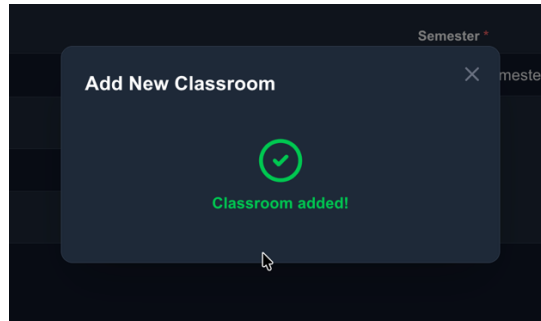
- **A. Test Cases:**
 - **Action:** Click "X" button -> Click "Cancel" in the modal.
 - **Expected Result:** The student remains in the list; no changes occur.
- **B. Test Procedure:**
 1. Click the Remove (X) button for a student.
 2. Click "Cancel" in the confirmation modal.
 3. Verify the student is still in the list.



Test Case 13 – Test Condition: Successfully create a new classroom via the Quick Add modal.

- **A. Test Cases:**
 - **Precondition:** User is on the Course Registration page.
 - **Inputs (Valid):**
 - Click the "+" button next to the Classroom dropdown.
 - Classroom Code: "LAB-5"
 - Capacity: 30
 - **Expected Result:**
 1. The system displays a success message: *"Classroom created successfully!"*.
 2. The modal closes automatically after a few seconds.
 3. The new classroom "LAB-5" is automatically added to the dropdown list and selected.
- **B. Test Procedure:**
 1. Click the "+" (Plus) icon next to the Classroom select box.
 2. Enter "LAB-5" in the *Classroom Code* field.
 3. Enter "30" in the *Capacity* field.
 4. Click the "Save Classroom" button.
 5. Verify the success message and check if "LAB-5" is selected in the main form.





8.3 Test Cases for Instructor Management Page

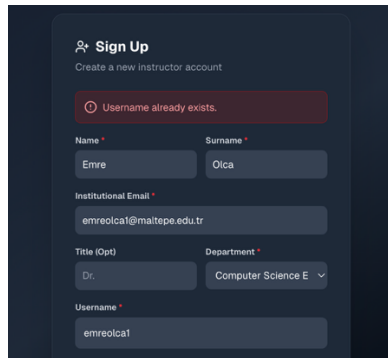
Source Files: backend/routes.py (/instructors POST)

Test Case 1 - Test Condition: An instructor should be created when all required fields (Name, Surname, Username, Password, Email, Department) are entered.

- **A. Test Cases:**
 - **Inputs (Valid):** Name: "Emre", Surname: "Olca", User: "emreolca", Pass: "1234", Email: "emreolca@edu.tr", Dept: "CSE" -> **Expected Result:** "Instructor created" message (HTTP 201).
- **B. Test Procedure:**
 1. Fill in the instructor addition form completely.
 2. Click the "Save" button.
 3. Observe the success message.

Test Case 2 - Test Condition: Username must be unique.

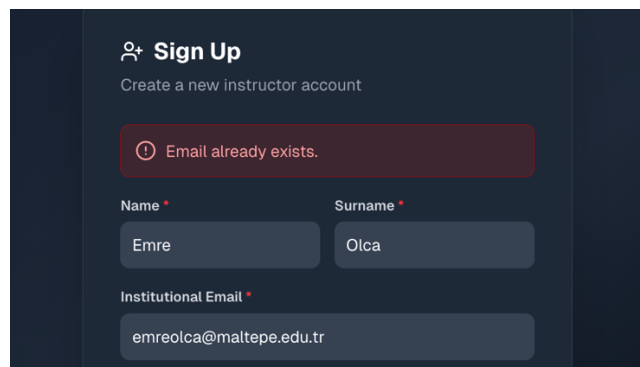
- **A. Test Cases:**
 - **Inputs (Invalid):** Username: "emreolca" (Existing) -> **Expected Result:** "Username or Email already exists" error.
- **B. Test Procedure:**
 1. Enter an existing username again.
 2. Submit the form.
 3. Verify the error message.



The screenshot shows a 'Sign Up' form for creating a new instructor account. At the top, there is a red error message: 'Username already exists.' Below the form fields, the 'Name' field contains 'Emre' and the 'Surname' field contains 'Olca'. The 'Institutional Email' field contains 'emreolca1@maltepe.edu.tr'. The 'Title (Opt)' field is set to 'Dr.' and the 'Department' dropdown is set to 'Computer Science E'. The 'Username' field contains 'emreolca1'.

Test Case 3 - Test Condition: Email address must be unique.

- **A. Test Cases:**
 - **Inputs (Invalid):** Username: "newTeacher", Email: "newteacher@edu.tr" (Existing) -> **Expected Result:** Conflict error (409).
- **B. Test Procedure:**
 1. Enter a different username but the same email address.
 2. Click the Save button.
 3. Check that an error is received.



The screenshot shows the same 'Sign Up' form. The 'Email already exists.' error message is still present. The 'Name' field contains 'Emre' and the 'Surname' field contains 'Olca'. The 'Institutional Email' field contains 'emreolca@maltepe.edu.tr'. The 'Title (Opt)' field is set to 'Dr.' and the 'Department' dropdown is set to 'Computer Science E'. The 'Username' field is empty.

Test Case 4 - Test Condition: Title field can be left blank (Optional).

- **A. Test Cases:**

- **Inputs (Valid):** Title: [Empty] -> **Expected Result:** Registration is successful.
- **B. Test Procedure:**
 1. Fill in the form without writing a title like "Dr." or "Prof.".
 2. Save.
 3. Verify that the system accepts it.

Sign Up
Create a new instructor account

Name * Surname *

Institutional Email *

Title (Opt) Department *

Dr. Select Department

Username *

Password *

Confirm Password *

Sign Up

Already have an account? Log in

Test Case 5 - Test Condition: Password field is mandatory.

- **A. Test Cases:**
 - **Inputs (Invalid):** Password: [Empty] -> **Expected Result:** "Missing required fields" error.
- **B. Test Procedure:**
 1. Leave the password field blank.
 2. Click the Save button.
 3. Verify the warning.

Şifre

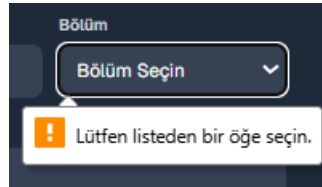
Şifre Tekrar

Lütfen bu alanı doldurun.

Test Case 6 - Test Condition: Instructor cannot be created without entering Department information.

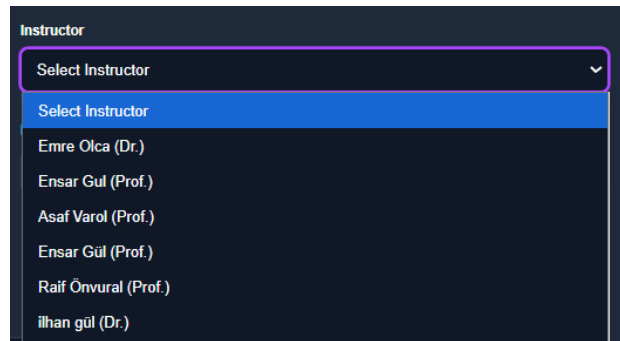
- **A. Test Cases:**
 - **Inputs (Invalid):** Department: [Empty] -> **Expected Result:** Error message.
- **B. Test Procedure:**

1. Submit the form without selecting/entering a department.
2. Check for the error message.



Test Case 7 - Test Condition: When an instructor is created, the "Instructor" role should be automatically assigned in the system.

- **A. Test Cases:**
 - **Inputs (Valid):** New Instructor Data -> **Expected Result:** Must be associated with "Instructor" in the `role` table in the database.
- **B. Test Procedure:**
 1. Complete the registration.
 2. Check that the role appears as "Instructor" or "Teacher" in the user list (if available) or database.



Test Case 8 - Test Condition: Name and Surname fields cannot be left blank.

- **A. Test Cases:**
 - **Inputs (Invalid):** Name: [Empty] -> **Expected Result:** Required field warning.
- **B. Test Procedure:**
 1. Clear the Name field.
 2. Click the Save button.
 3. Verify that the operation does not proceed.

9. Unit Testing Implementation

9.1 Frontend Tests

9.1.1 Unit Tests for Student Module

Student Registration Form Tests

Test 1: Required Fields Validation

Tests that all mandatory form fields are rendered and have the `required` attribute. Verifies:

- Student Number input field
- Name input field
- Surname input field
- Department select dropdown
- Class select dropdown

Test 2: Invalid Format Validation

Tests form validation when invalid data formats are entered. The test:

- Enters invalid student number (contains letters: "ABC123")
- Fills other required fields with valid data
- Submits the form
- Verifies that error messages are displayed for invalid formats

Test 3: Photo Upload Validation

Tests that photo upload is required for new student registrations. The test:

- Fills all form fields with valid data
- Does not upload a photo
- Submits the form
- Verifies that a photo required error message is displayed

Source Code:

```

import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import StudentRegistrationPage from '../page'

// Mock fetch
global.fetch = jest.fn()

// Mock Next.js Image component
jest.mock('next/image', () => ({
  __esModule: true,
  default: (props: any) => {
    const { priority, ...restProps } = props
    return <img {...restProps} />
  },
}))

describe('Student Registration Form - Unit Tests', () => {
  beforeEach(() => {
    (fetch as jest.Mock).mockClear()
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
  })

  test('1. Zorunlu form alanları render edilmeli ve required attribute\'a sahip olmalı', async () => {
    render(<StudentRegistrationPage />)

    await waitFor(() => {
      expect(screen.getByPlaceholderText(/2024001|e\.g\./i)).toBeInTheDocument()
    })

    const studentNumberInput = screen.getByPlaceholderText(/2024001|e\.g\./i)
    expect(studentNumberInput).toBeInTheDocument()
    expect(studentNumberInput).toHaveAttribute('required')

    const nameInput = screen.getByPlaceholderText(/first name|ad/i)
    expect(nameInput).toBeInTheDocument()
    expect(nameInput).toHaveAttribute('required')

    const surnameInput = screen.getByPlaceholderText(/last name|soyad/i)
    expect(surnameInput).toBeInTheDocument()
    expect(surnameInput).toHaveAttribute('required')

    const allSelects = screen.getAllByRole('combobox')
    const departmentSelect = Array.from(allSelects).find(
      (select) => (select as HTMLSelectElement).name === 'department'
    ) as HTMLSelectElement
    expect(departmentSelect).toHaveAttribute('required')
  })
})

```

```

const classSelect = Array.from(allSelects).find(
  (select) => (select as HTMLSelectElement).name === 'class'
) as HTMLSelectElement
expect(classSelect).toHaveAttribute('required')
})

test('2. Geçersiz formatlarda veri girildiğinde hata mesajları gösterilmeli', async () => {
  const user = userEvent.setup()
  render(<StudentRegistrationPage />)

  await waitFor(() => {
    expect(screen.getByPlaceholderText(/2024001|e\.g\./i)).toBeInTheDocument()
  })

  const studentNumberInput = screen.getByPlaceholderText(/2024001|e\.g\./i)
  await user.clear(studentNumberInput)
  await user.type(studentNumberInput, 'ABC123')

  const nameInput = screen.getByPlaceholderText(/first name|ad/i)
  await user.type(nameInput, 'Test')

  const surnameInput = screen.getByPlaceholderText(/last name|soyad/i)
  await user.type(surnameInput, 'User')

  const allSelects = screen.getAllByRole('combobox')
  const departmentSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'department'
  ) as HTMLSelectElement
  await user.selectOptions(departmentSelect, 'CSE')

  const classSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'class'
  ) as HTMLSelectElement
  await user.selectOptions(classSelect, '1')

  const submitButton = screen.getByRole('button', { name: /register student/i })
  await user.click(submitButton)

  await waitFor(() => {
    const errorMessages = screen.queryAllByText((content, element) => {
      const hasErrorClass = element?.classList?.contains('text-red-400')
      const hasFormatError = /digits|format|valid/i.test(content)
      return hasErrorClass && hasFormatError
    })
    expect(errorMessages.length).toBeGreaterThan(0)
  }, { timeout: 2000 })
})

test('3. Fotoğraf yükleme validasyonu çalışmalı - Yeni kayıt için fotoğraf zorunlu', async () => {
  const user = userEvent.setup()
  render(<StudentRegistrationPage />)

```

```

await waitFor(() => {
  expect(screen.getByPlaceholderText(/2024001|e\.g\./i)).toBeInTheDocument()
})

const studentNumberInput = screen.getByPlaceholderText(/2024001|e\.g\./i)
await user.type(studentNumberInput, '2024001')

const nameInput = screen.getByPlaceholderText(/first name|ad/i)
await user.type(nameInput, 'Test')

const surnameInput = screen.getByPlaceholderText(/last name|soyad/i)
await user.type(surnameInput, 'User')

const allSelects = screen.getAllByRole('combobox')
const departmentSelect = Array.from(allSelects).find(
  (select) => (select as HTMLSelectElement).name === 'department'
) as HTMLSelectElement
await user.selectOptions(departmentSelect, 'CSE')

const classSelect = Array.from(allSelects).find(
  (select) => (select as HTMLSelectElement).name === 'class'
) as HTMLSelectElement
await user.selectOptions(classSelect, '1')

const submitButton = screen.getByRole('button', { name: /register student/i })
await user.click(submitButton)

await waitFor(() => {
  const photoError = screen.queryByText((content, element) => {
    const hasErrorClass = element?.classList?.contains('text-red-400')
    const hasPhotoError = /photo|foto/i.test(content) && /required/i.test(content)
    return hasErrorClass && hasPhotoError
  })
  expect(photoError).toBeInTheDocument()
}, { timeout: 2000 })
})
})

```

9.1.2 Unit Tests for Course Module

Course Registration Form Tests

Test 1: Required Fields Validation

Tests that all mandatory form fields are rendered and have the `required` attribute. Verifies:

- Course Code input field
- Course Name input field
- Semester select dropdown
- Academic Year select dropdown

Test 2: Invalid Format Validation

Tests form validation when invalid data formats are entered. The test:

- Enters invalid course code (contains special characters: "###INVALID###")
- Enters negative credits value (-5)
- Fills other required fields
- Submits the form
- Verifies that validation prevents submission with invalid data

Test 3: Form Submission API Call

Tests that submitting a valid form triggers an API call. The test:

- Fills all form fields with valid data
- Submits the form
- Verifies that the fetch API is called

Source Code:

```
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import CourseRegistrationPage from '../page'

// Mock fetch
global.fetch = jest.fn()

// Mock Next.js Image component
jest.mock('next/image', () => ({
  __esModule: true,
  default: (props: any) => {
    const { priority, ...restProps } = props
    return <img {...restProps} />
  },
}))

describe('Course Registration Form - Unit Tests', () => {
  beforeEach(() => {
    (fetch as jest.Mock).mockClear()
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
  })
})
```

```

test('1. Zorunlu form alanları render edilmeli ve required attribute\'a sahip olmalı', async () => {
  render(<CourseRegistrationPage />)

  await waitFor(() => {
    expect(screen.getByPlaceholderText(/SE342|course code/i)).toBeInTheDocument()
  })

  const courseCodeInput = screen.getByPlaceholderText(/SE342|course code/i)
  expect(courseCodeInput).toBeInTheDocument()
  expect(courseCodeInput).toHaveAttribute('required')

  const courseNameInput = screen.getByPlaceholderText(/software engineering|course name/i)
  expect(courseNameInput).toBeInTheDocument()
  expect(courseNameInput).toHaveAttribute('required')

  const allSelects = screen.getAllByRole('combobox')
  const semesterSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'semester'
  ) as HTMLSelectElement
  expect(semesterSelect).toHaveAttribute('required')

  const academicYearSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'academicYear'
  ) as HTMLSelectElement
  expect(academicYearSelect).toHaveAttribute('required')
})

test('2. Geçersiz formatlarda veri girildiğinde hata mesajları gösterilmeli', async () => {
  const user = userEvent.setup()
  render(<CourseRegistrationPage />)

  await waitFor(() => {
    expect(screen.getByPlaceholderText(/SE342|course code/i)).toBeInTheDocument()
  })

  const courseCodeInput = screen.getByPlaceholderText(/SE342|course code/i)
  await user.clear(courseCodeInput)
  await user.type(courseCodeInput, '###INVALID###')

  const courseNameInput = screen.getByPlaceholderText(/software engineering|course name/i)
  await user.type(courseNameInput, 'Test Course')

  // Credits input'unu name attribute ile bul
  const allInputs = screen.getAllByRole('spinbutton')
  const creditsInput = allInputs.find(
    (input) => (input as HTMLInputElement).name === 'credits'
  ) as HTMLInputElement
  if (creditsInput) {
    await user.clear(creditsInput)
    await user.type(creditsInput, '-5')
  }
}

```

```

const allSelects = screen.getAllByRole('combobox')
const semesterSelect = Array.from(allSelects).find(
  (select) => (select as HTMLSelectElement).name === 'semester'
) as HTMLSelectElement
await user.selectOptions(semesterSelect, 'Fall')

const academicYearSelect = Array.from(allSelects).find(
  (select) => (select as HTMLSelectElement).name === 'academicYear'
) as HTMLSelectElement
// Academic year formatı "2024-2025" gibi, ilk geçerli option'ı seç
if (academicYearSelect && academicYearSelect.options.length > 1) {
  await user.selectOptions(academicYearSelect, academicYearSelect.options[1].value)
}

// Form submit et
const submitButton = screen.getByRole('button', { name: /register course/i })
await user.click(submitButton)

// Validasyon çalışmalı - geçersiz veri girişi yapıldığını kontrol et
// Hata mesajları gösterilebilir veya form submit edilmemiş olabilir
await waitFor(() => {
  // Geçersiz kurs kodu hala input'ta mı kontrol et
  expect(courseCodeInput).toHaveValue('###INVALID###')
  // Form submit edilmiş olabilir ama validasyon çalışmış olmalı
  expect(submitButton).toBeInTheDocument()
}, { timeout: 2000 })
})

test('3. Form submit edildiğinde API çağrısı yapılmalı', async () => {
  const user = userEvent.setup()
  render(<CourseRegistrationPage />)

  await waitFor(() => {
    expect(screen.getByPlaceholderText(/SE342|course code/i)).toBeInTheDocument()
  })

  const courseCodeInput = screen.getByPlaceholderText(/SE342|course code/i)
  await user.type(courseCodeInput, 'SE342')

  const courseNameInput = screen.getByPlaceholderText(/software engineering|course name/i)
  await user.type(courseNameInput, 'Software Engineering')

  const allSelects = screen.getAllByRole('combobox')
  const semesterSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'semester'
  ) as HTMLSelectElement
  await user.selectOptions(semesterSelect, 'Fall')

  const academicYearSelect = Array.from(allSelects).find(
    (select) => (select as HTMLSelectElement).name === 'academicYear'

```

```

) as HTMLSelectElement
// Academic year format "2024-2025" gibi, ilk geçerli option'ı seç
if (academicYearSelect && academicYearSelect.options.length > 1) {
  await user.selectOptions(academicYearSelect, academicYearSelect.options[1].value)
}

;(fetch as jest.Mock).mockResolvedValueOnce({
  ok: true,
  json: async () => ({ success: true }),
})

const submitButton = screen.getByRole('button', { name: /register course/i })
await user.click(submitButton)

await waitFor(() => {
  expect(fetch).toHaveBeenCalled()
}, { timeout: 2000 })
})
})

```

9.1.3 Unit Tests for Enrollment Module

Course Enrollment Form Tests

Test 1: Course Selection Rendering

Tests that the course selection dropdown is rendered correctly. Verifies:

- Select Course label is present
- Course selection dropdown (combobox) is rendered

Test 2: API Call on Course Selection

Tests that selecting a course triggers an API call to fetch student list. The test:

- Renders the enrollment page
- Selects a course from the dropdown
- Verifies that the fetch API is called to retrieve student data

Test 3: Form Elements Rendering

Tests that all form elements are rendered correctly. Verifies:

- Page title (H1 element) is present
- Course selection dropdown is rendered
- Select Course label is present

Source Code:

```

import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'

```

```

import CourseEnrollmentPage from '../page'

// Mock fetch
global.fetch = jest.fn()

// Mock Next.js Image component
jest.mock('next/image', () => ({
  __esModule: true,
  default: (props: any) => {
    const { priority, ...restProps } = props
    return <img {...restProps} />
  },
}))

describe('Course Enrollment Form - Unit Tests', () => {
  beforeEach(() => {
    (fetch as jest.Mock).mockClear()
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
  })

  test('1. Kurs seçimi render edilmeli', async () => {
    render(<CourseEnrollmentPage />)

    await waitFor(() => {
      const allSelects = screen.getAllByRole('combobox')
      expect(allSelects.length).toBeGreaterThan(0)
    })

    // Select Course label'ı var mı kontrol et (label element olarak)
    const selectLabels = screen.getAllByText(/select course|kurs seçin/i)
    const labelElement = selectLabels.find(el => el.tagName === 'LABEL')
    expect(labelElement).toBeInTheDocument()

    // Select element'ini bul
    const allSelects = screen.getAllByRole('combobox')
    expect(allSelects.length).toBeGreaterThan(0)
  })

  test('2. Kurs seçildiğinde öğrenci listesi için API çağrısı yapılmalı', async () => {
    const user = userEvent.setup()

    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [{ id: 1, code: 'SE342', name: 'Software Engineering' }],
    })
  })
})

```

```

    })
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })
    ;(fetch as jest.Mock).mockResolvedValueOnce({
      ok: true,
      json: async () => [],
    })

    render(<CourseEnrollmentPage />)

    await waitFor(() => {
      const allSelects = screen.getAllByRole('combobox')
      expect(allSelects.length).toBeGreaterThan(0)
    })

    // Select element'ini bul - kurs seçimi için value değişikliği yap
    const allSelects = screen.getAllByRole('combobox')
    const courseSelect = allSelects[0] as HTMLSelectElement

    // Select'in value'sunu direkt değiştir (option yoksa)
    if (courseSelect.options.length > 1) {
      await user.selectOptions(courseSelect, courseSelect.options[1].value)
    } else {
      // Eğer option yoksa, sadece API çağrısının yapıldığını kontrol et
      expect(fetch).toHaveBeenCalled()
    }

    // API çağrılarının yapıldığını kontrol et
    await waitFor(() => {
      expect(fetch).toHaveBeenCalled()
    }, { timeout: 2000 })
  })

  test('3. Form elementleri render edilmeli', async () => {
    render(<CourseEnrollmentPage />)

    await waitFor(() => {
      const allSelects = screen.getAllByRole('combobox')
      expect(allSelects.length).toBeGreaterThan(0)
    })

    // Başlık kontrolü - h1 elementi olarak
    const titles = screen.getAllByText(/enroll students|kursa öğrenci/i)
    const h1Title = titles.find(el => el.tagName === 'H1')
    expect(h1Title).toBeInTheDocument()

    // Select element var mı
    const allSelects = screen.getAllByRole('combobox')
    expect(allSelects.length).toBeGreaterThan(0)
  })

```

```

// Select Course label'i var mı (label element olarak)
const selectLabels = screen.getAllByText(/select course|kurs seçin/i)
const labelElement = selectLabels.find(el => el.tagName === 'LABEL')
expect(labelElement).toBeInTheDocument()
})
})

```

Test Results:

```

PS C:\Users\cifte\OneDrive\Desktop\aaS\SE342-AttendanceSystem\frontend> npm test

> frontend@0.1.0 test
> jest

[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: `
npm i baseline-browser-mapping@latest -D`
[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: `
npm i baseline-browser-mapping@latest -D`
[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: `
npm i baseline-browser-mapping@latest -D`
[baseline-browser-mapping] The data in this module is over two months old. To ensure accurate Baseline data, please update: `
PASS app/course-enrollment/__tests__/page.test.tsx
PASS app/student-registration/__tests__/page.test.tsx
PASS app/course-registration/__tests__/page.test.tsx

Test Suites: 3 passed, 3 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 3.479 s
Ran all test suites.

```

9.2 Backend Tests

9.2.1 Unit Tests for Student Module

```

import unittest
import json
import base64
from io import BytesIO
from app import app, db
from models import Role, User, Student, Department, Classroom

class TestStudentRegistration(unittest.TestCase):

    def setUp(self):
        """Her testten önce çalışır: Geçici veritabanı kurar ve test verisi basar."""
        app.config['TESTING'] = True
        app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///memory:' # RAM üzerinde DB

```

```

self.app = app.test_client()

with app.app_context():
    db.create_all()
    self.create_dummy_data()

def tearDown(self):
    """Her testten sonra çalışır: Veritabanını temizler."""
    with app.app_context():
        db.session.remove()
        db.drop_all()

def create_dummy_data(self):
    """Testler için gerekli asgari verileri oluşturur."""
    # 1. Role ve Department
    role = Role(id="role_1", name="Student")
    dept = Department(id="dept_1", name="Computer Engineering")

    # 2. Sınıf (kapasite kontrolü için)
    classroom = Classroom(id="class_1", classroom_code="A-101", capacity=30)

    db.session.add_all([role, dept, classroom])
    db.session.commit()

def get_test_image_base64(self):
    """Test için base64 encoded fake image oluşturur."""
    # 1x1 pixel PNG (en küçük geçerli resim)
    img_data =
b'\x89PNG\r\n\x1a\n\x00\x00\x00\rIHDR\x00\x00\x00\x01\x00\x00\x00\x01\x08\x02\x00\x00\x00\x9
0wS\xde'
    return base64.b64encode(img_data).decode('utf-8')

#
=====
# TEST SENARYOLARI
#
=====

def test_scenario_1_successful_registration(self):
    """Senaryo 1: Başarılı Kayıt (Happy Path)"""
    # Arrange
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

```

```

# Act
response = self.app.post('/api/students',
                        data=json.dumps(payload),
                        content_type='application/json')

# Assert
self.assertEqual(response.status_code, 201)
response_data = json.loads(response.data)
self.assertIn("success", response_data)
self.assertTrue(response_data["success"])

# Veritabanında gerçekten oluştu mu?
with app.app_context():
    student_count = Student.query.count()
    self.assertEqual(student_count, 1)

    student = Student.query.filter_by(student_number=2024001).first()
    self.assertIsNotNone(student)
    self.assertEqual(student.email, "ahmet@test.com")

def test_scenario_2_missing_required_fields(self):
    """Senaryo 2: Eksik Veri Kontrolü - Zorunlu alan eksik"""
    # Arrange - first_name eksik
    payload = {
        "student_number": "2024001",
        # "first_name": "Ahmet", # EKSİK!
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

    # Act
    response = self.app.post('/api/students',
                            data=json.dumps(payload),
                            content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 400)
    response_data = json.loads(response.data)
    self.assertIn("Missing required fields", response_data.get("error", ""))

def test_scenario_3_invalid_student_number_format(self):
    """Senaryo 3: Geçersiz Format - Öğrenci numarası yanlış formatta"""
    # Arrange - Öğrenci numarası harf içeriyor
    payload = {
        "student_number": "ABC123", # HATALI FORMAT!
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",

```

```

        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

    # Act
    response = self.app.post('/api/students',
                             data=json.dumps(payload),
                             content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 400)
    response_data = json.loads(response.data)
    self.assertIn("Student number must be 7 digits", response_data.get("error", ""))

def test_scenario_4_missing_photo(self):
    """Senaryo 4: Fotoğraf Eksik - Fotoğraf zorunlu alan"""
    # Arrange - photo eksik
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "ahmet@test.com"
        # "photo": self.get_test_image_base64() # EKSIK!
    }

    # Act
    response = self.app.post('/api/students',
                             data=json.dumps(payload),
                             content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 400)
    response_data = json.loads(response.data)
    self.assertIn("Photo is required", response_data.get("error", ""))

def test_scenario_5_duplicate_student_number(self):
    """Senaryo 5: Mükerrer Kayıt - Aynı öğrenci numarasıyla iki kayıt"""
    # Arrange
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

```

```

# Act - İlk kayıt başarılı olsun
first_response = self.app.post('/api/students',
                                data=json.dumps(payload),
                                content_type='application/json')
self.assertEqual(first_response.status_code, 201)

# İkinci kayıt denemesi (aynı student_number)
second_response = self.app.post('/api/students',
                                data=json.dumps(payload),
                                content_type='application/json')

# Assert - 409 Conflict dönmeli
self.assertEqual(second_response.status_code, 409)
response_data = json.loads(second_response.data)
self.assertIn("Student number already exists", response_data.get("error", ""))

def test_scenario_6_invalid_department_id(self):
    """Senaryo 6: Olmayan Department - Geçersiz department_id"""
    # Arrange - Olmayan department
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "olmayan_dept_999", # GEÇERSİZ!
        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

    # Act
    response = self.app.post('/api/students',
                              data=json.dumps(payload),
                              content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 404)
    response_data = json.loads(response.data)
    self.assertIn("Department not found", response_data.get("error", ""))

def test_scenario_7_invalid_email_format(self):
    """Senaryo 7: Geçersiz E-posta Formatı"""
    # Arrange - Hatalı email
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "invalid-email-format", # HATALI FORMAT!
        "photo": self.get_test_image_base64()
    }

```

```

# Act
response = self.app.post('/api/students',
                        data=json.dumps(payload),
                        content_type='application/json')

# Assert
self.assertEqual(response.status_code, 400)
response_data = json.loads(response.data)
self.assertIn("Invalid email format", response_data.get("error", ""))

def test_scenario_8_get_student_by_id(self):
    """Senaryo 8: Mevcut Öğrenci Bilgisini Getirme"""
    # Arrange - Önce bir öğrenci kaydet
    payload = {
        "student_number": "2024001",
        "first_name": "Ahmet",
        "last_name": "Yılmaz",
        "department_id": "dept_1",
        "class_id": "class_1",
        "email": "ahmet@test.com",
        "photo": self.get_test_image_base64()
    }

    create_response = self.app.post('/api/students',
                                    data=json.dumps(payload),
                                    content_type='application/json')

    created_student_id = json.loads(create_response.data)["data"]["id"]

    # Act - Öğrenci bilgisini getir
    response = self.app.get(f'/api/students/{created_student_id}')

    # Assert
    self.assertEqual(response.status_code, 200)
    response_data = json.loads(response.data)
    self.assertEqual(response_data["data"]["student_number"], "2024001")
    self.assertEqual(response_data["data"]["first_name"], "Ahmet")

def test_scenario_9_get_nonexistent_student(self):
    """Senaryo 9: Olmayan Öğrenci ID'si ile Getirme"""
    # Act - Olmayan ID ile sorgu
    response = self.app.get('/api/students/nonexistent_id_999')

    # Assert
    self.assertEqual(response.status_code, 404)
    response_data = json.loads(response.data)
    self.assertIn("Student not found", response_data.get("error", ""))

if __name__ == "__main__":

```

```
unittest.main()
```

9.2.2 Unit Tests for Course Module

```
import unittest
import json
from app import app, db
from models import Role, User, Department, Classroom, Course

class TestCourseRegistration(unittest.TestCase):

    def setUp(self):
        """Her testten önce çalışır: Geçici veritabanı kurar ve test verisi basar."""
        app.config['TESTING'] = True
        app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///memory:' # RAM üzerinde DB
        self.app = app.test_client()

        with app.app_context():
            db.create_all()
            self.create_dummy_data()

    def tearDown(self):
        """Her testten sonra çalışır: Veritabanını temizler."""
        with app.app_context():
            db.session.remove()
            db.drop_all()

    def create_dummy_data(self):
        """Testler için gerekli asgari verileri oluşturur."""
        # 1. Department ve Classroom
        dept = Department(id="dept_1", name="Computer Engineering")
        classroom = Classroom(id="class_1", classroom_code="A-101", capacity=30)

        db.session.add_all([dept, classroom])
        db.session.commit()

#
=====
# TEST SENARYOLARI
#
=====

def test_scenario_1_successful_registration(self):
    """Senaryo 1: Başarılı Kayıt (Happy Path)"""
    # Arrange
    payload = {
        "course_code": "SE342",
        "course_name": "Software Engineering",
        "credits": 3,
        "semester": "Fall",
```

```

        "academic_year": "2024-2025",
        "classroom_id": "class_1",
        "department_id": "dept_1"
    }

    # Act
    response = self.app.post('/api/courses',
                             data=json.dumps(payload),
                             content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 201)
    response_data = json.loads(response.data)
    self.assertIn("success", response_data)
    self.assertTrue(response_data["success"])

    # Veritabanında gerçekten oluştu mu?
    with app.app_context():
        course_count = Course.query.count()
        self.assertEqual(course_count, 1)

        course = Course.query.filter_by(course_code="SE342").first()
        self.assertIsNotNone(course)
        self.assertEqual(course.course_name, "Software Engineering")
        self.assertEqual(course.credits, 3)

def test_scenario_2_missing_required_fields(self):
    """Senaryo 2: Eksik Zorunlu Alan - course_name eksik"""
    # Arrange - course_name eksik
    payload = {
        "course_code": "SE342",
        # "course_name": "Software Engineering", # EKSİK!
        "credits": 3,
        "semester": "Fall",
        "academic_year": "2024-2025",
        "classroom_id": "class_1",
        "department_id": "dept_1"
    }

    # Act
    response = self.app.post('/api/courses',
                             data=json.dumps(payload),
                             content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 400)
    response_data = json.loads(response.data)
    self.assertIn("Missing required fields", response_data.get("error", ""))

def test_scenario_3_invalid_course_code_format(self):
    """Senaryo 3: Geçersiz Format - Kurs kodu özel karakter içeriyor"""

```

```

# Arrange - Kurs kodu geçersiz
payload = {
    "course_code": "###INVALID###", # HATALI FORMAT!
    "course_name": "Software Engineering",
    "credits": 3,
    "semester": "Fall",
    "academic_year": "2024-2025",
    "classroom_id": "class_1",
    "department_id": "dept_1"
}

# Act
response = self.app.post('/api/courses',
                        data=json.dumps(payload),
                        content_type='application/json')

# Assert
self.assertEqual(response.status_code, 400)
response_data = json.loads(response.data)
self.assertIn("Invalid course code format", response_data.get("error", ""))

def test_scenario_4_negative_credits(self):
    """Senaryo 4: Negatif Kredi - Kredi değeri negatif olamaz"""
    # Arrange - Negatif kredi
    payload = {
        "course_code": "SE342",
        "course_name": "Software Engineering",
        "credits": -5, # NEGATIF DEĞER!
        "semester": "Fall",
        "academic_year": "2024-2025",
        "classroom_id": "class_1",
        "department_id": "dept_1"
    }

    # Act
    response = self.app.post('/api/courses',
                            data=json.dumps(payload),
                            content_type='application/json')

    # Assert
    self.assertEqual(response.status_code, 400)
    response_data = json.loads(response.data)
    self.assertIn("Credits must be positive", response_data.get("error", ""))

def test_scenario_5_duplicate_course_code(self):
    """Senaryo 5: Duplicate Kurs Kodu - Aynı kurs koduyla iki kayıt yapılamaz"""
    # Arrange
    payload = {
        "course_code": "SE342",
        "course_name": "Software Engineering",
        "credits": 3,

```

```

        "semester": "Fall",
        "academic_year": "2024-2025",
        "classroom_id": "class_1",
        "department_id": "dept_1"
    }

    # Act - İlk kayıt başarılı olsun
    first_response = self.app.post('/api/courses',
                                   data=json.dumps(payload),
                                   content_type='application/json')
    self.assertEqual(first_response.status_code, 201)

    # İkinci kayıt denemesi (aynı course_code)
    second_response = self.app.post('/api/courses',
                                    data=json.dumps(payload),
                                    content_type='application/json')

    # Assert - 409 Conflict dönmeli
    self.assertEqual(second_response.status_code, 409)
    response_data = json.loads(second_response.data)
    self.assertIn("Course code already exists", response_data.get("error", ""))

if __name__ == "__main__":
    unittest.main()

```

9.2.3 Unit Tests for Enrollment Module

```

import unittest
import json
import warnings # Uyarıları yönetmek için
from app import app, db
from models import (
    Role, User, Student, Department, Classroom, Course, Enrollment
)
from sqlalchemy.exc import LegacyAPIWarning

class TestEnrollmentMultiScenarios(unittest.TestCase):

    def setUp(self):
        """Her testten önce çalışır."""
        # 1. GEREKSİZ UYARILARI SUSTUR (Temiz çıktı için)
        warnings.simplefilter("ignore", category=LegacyAPIWarning)
        warnings.simplefilter("ignore", category=DeprecationWarning)

        # 2. Konfigürasyon
        app.config['TESTING'] = True
        app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///memory:'

```

```

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# 3. Context ve DB Hazırlığı
self.app_context = app.app_context()
self.app_context.push()

# PostgreSQL bağlantısını kopar, SQLite'a zorla
db.engine.dispose()

db.create_all()
self.create_dummy_data()

self.client = app.test_client()

def tearDown(self):
    """Temizlik"""
    db.session.remove()
    db.drop_all()
    self.app_context.pop()

def get_or_create(self, model, **kwargs):
    """Veri varsa getirir, yoksa oluşturur."""
    instance = model.query.filter_by(**kwargs).first()
    if instance:
        return instance
    else:
        instance = model(**kwargs)
        db.session.add(instance)
        db.session.commit()
        return instance

def create_dummy_data(self):
    """Test verilerini oluşturur (Legacy Warning düzeltildi)"""
    try:
        # SQLAlchemy 2.0 Uyumlu: db.session.get() kullanıldı

        # 1. Role & Department
        role = self.get_or_create(Role, id="role_1", name="Student")

        dept = Department.query.filter_by(name="Computer Engineering").first()
        if not dept:
            dept = Department(id="dept_1", name="Computer Engineering")
            db.session.add(dept)
        else:
            dept_id = dept.id
        db.session.commit()

        # 2. Kullanıcılar (Query.get yerine db.session.get kullandık)
        if not db.session.get(User, "user_1"):
            user1 = User(id="user_1", first_name="Ali", last_name="Yilmaz",
                        password="pass", role_id="role_1")

```

```

db.session.add(user1)

if not db.session.get(Student, "student_1"):
    student1 = Student(id="student_1", user_id="user_1", student_number=1001,
                       email="ali@test.com", department_id=dept.id)
    db.session.add(student1)

if not db.session.get(User, "user_2"):
    user2 = User(id="user_2", first_name="Ayse", last_name="Demir",
                 password="pass", role_id="role_1")
    db.session.add(user2)

if not db.session.get(Student, "student_2"):
    student2 = Student(id="student_2", user_id="user_2", student_number=1002,
                       email="ayse@test.com", department_id=dept.id)
    db.session.add(student2)

# 3. Sınıf & Ders
if not db.session.get(Classroom, "class_1"):
    classroom = Classroom(id="class_1", classroom_code="A-101", capacity=1)
    db.session.add(classroom)

if not db.session.get(Course, "course_1"):
    course = Course(id="course_1", course_code="CS101", course_name="Intro to CS",
                    credits=3, classroom_id="class_1")
    db.session.add(course)

db.session.commit()

except Exception as e:
    db.session.rollback()
    print(f"Veri hatası: {e}")

#
=====
# TEST SENARYOLARI
#
=====

def test_enroll_student_success(self):
    """Senaryo 1: Başarılı Kayıt ....."""
    payload = {"student_id": "student_1", "course_id": "course_1", "year": 2024, "term": "Fall"}
    response = self.client.post('/api/enrollments', data=json.dumps(payload),
                                content_type='application/json')
    self.assertEqual(response.status_code, 201)

def test_missing_fields(self):
    """Senaryo 2: Eksik Veri Kontrolü ....."""
    payload = { "course_id": "course_1" }
    response = self.client.post('/api/enrollments', data=json.dumps(payload),
                                content_type='application/json')

```

```

self.assertEqual(response.status_code, 400)

def test_course_not_found(self):
    """Senaryo 3: Olmayan Ders Kontrolü ....."""
    payload = {"student_id": "student_1", "course_id": "999", "year": 2024, "term": "Fall"}
    response = self.client.post('/api/enrollments', data=json.dumps(payload),
content_type='application/json')
    self.assertEqual(response.status_code, 404)

def test_duplicate_enrollment(self):
    """Senaryo 4: Mükerrer Kayıt (Aynı ders) ....."""
    payload = {"student_id": "student_1", "course_id": "course_1", "year": 2024, "term": "Fall"}
    self.client.post('/api/enrollments', data=json.dumps(payload), content_type='application/json')
    response = self.client.post('/api/enrollments', data=json.dumps(payload),
content_type='application/json')
    self.assertEqual(response.status_code, 409)

def test_classroom_capacity_full(self):
    """Senaryo 5: Kapasite Doluluğu (Full Class) ....."""
    # 1. Öğrenci (Kapasite doldu)
    payload_1 = {"student_id": "student_1", "course_id": "course_1", "year": 2024, "term": "Fall"}
    self.client.post('/api/enrollments', data=json.dumps(payload_1), content_type='application/json')

    # 2. Öğrenci (Hata vermeli)
    payload_2 = {"student_id": "student_2", "course_id": "course_1", "year": 2024, "term": "Fall"}
    response = self.client.post('/api/enrollments', data=json.dumps(payload_2),
content_type='application/json')
    self.assertEqual(response.status_code, 409)

if __name__ == "__main__":
    # verbosity=2 sayesinde çıktı "ok" şeklinde detaylı görünür
    unittest.main(verbosity=2)

```

```

test_classroom_capacity_full (__main__.TestEnrollmentMultiScenarios.test_classroom_capacity_full)
Senaryo 5: Kapasite Doluluğu (Full Class) ..... ok
test_course_not_found (__main__.TestEnrollmentMultiScenarios.test_course_not_found)
Senaryo 3: Olmayan Ders Kontrolü ..... ok
test_duplicate_enrollment (__main__.TestEnrollmentMultiScenarios.test_duplicate_enrollment)
Senaryo 4: Mükerrer Kayıt (Aynı ders) ..... ok
test_enroll_student_success (__main__.TestEnrollmentMultiScenarios.test_enroll_student_success)
Senaryo 1: Başarılı Kayıt ..... ok
test_missing_fields (__main__.TestEnrollmentMultiScenarios.test_missing_fields)
Senaryo 2: Eksik Veri Kontrolü ..... ok

-----
Ran 5 tests in 23.912s

OK

```

10. Automatic Attendance System Based on Face Recognition

10.1 Technologies and Libraries Used

1. YOLO (You Only Look Once) – Ultralytics

- Purpose: Person detection
- Model: YOLOv8n (Nano)
- Description: Used to detect humans in real-time video streams.
- Library: ultralytics >= 8.3.0

2. SORT (Simple Online and Realtime Tracking)

- Purpose: Object tracking
- Description: Tracks detected individuals across consecutive video frames.
- Algorithm: Kalman Filter + Hungarian Algorithm
- Library: filterpy >= 1.4.5 (for Kalman Filter)

3. OpenCV (Computer Vision Library)

- Purpose: Video processing and computer vision tasks
- Usage:
 - Video capture (cv2.VideoCapture)
 - Image preprocessing (resizing, color conversion)
 - Face detection using Haar Cascade
 - Histogram-based feature extraction
- Library: opencv-python >= 4.12.0

4. NumPy

- Purpose: Numerical computations
- Description: Used for matrix operations and array-based processing.
- Library: numpy >= 2.2.0

5. SciPy

- Purpose: Scientific computing and optimization
- Description: Used for optimization tasks, including the Hungarian algorithm.
- Library: scipy >= 1.16.0

6. PyTorch

- Purpose: Deep learning framework
- Description: Provides the runtime environment for the YOLO model.
- Libraries:
 - torch >= 2.9.0

- torchvision $\geq$ 0.24.0

7. Pillow (PIL)

- Purpose: Image handling
- Description: Used for loading and basic image processing operations.
- Library: pillow $\geq$ 12.0.0